

The `proofgraph` package

Pierre Senellart
`pierre@senellart.com`

2026/07/28 v1.0.1-dev

Abstract

The `proofgraph` package automatically produces a graph of the dependencies between the results (theorems, lemmas, propositions) of a mathematical article. It infers an edge from one result to another whenever the proof of the former refers to the latter through an ordinary cross-reference, so that most dependencies need no manual annotation; commands are provided for those a visible reference does not capture. The graph is written as a `Graphviz .dot` file, which can be rendered externally or, optionally, embedded into the document automatically.

1 Introduction

When writing or reading a mathematical article, it is often useful to visualise how the results depend on one another: which lemma is used in the proof of which theorem, and so on. `proofgraph` builds such a dependency graph automatically. Each theorem-like environment becomes a node; each cross-reference (`\ref`, `\cref`...) appearing *inside a proof* becomes an edge from the proved result to the referred-to result. Usually no manual annotation is required, and commands are provided for the dependencies a visible reference does not capture.

The package is best explained by a small example of its use. This manual is itself a `proofgraph` document, so the results stated below are typeset with the package enabled, and the graph shown at the end of this section is the one they produce.

Load `proofgraph` in the preamble, before the `\newtheorem` commands that declare your theorem-like environments (so it sees them as they are created). Here, as in this manual, we pass three options: `autorun` renders the graph with `Graphviz` and embeds it automatically (it needs shell-escape); `cite` captures the `\cites` made inside proofs as dependencies on external work; and `citelabel=tag` labels those citation nodes by their printed tag:

```
\usepackage[autorun,cite,citelabel=tag]{proofgraph}
\newtheorem{theorem}{Theorem}
\newtheorem{lemma}{Lemma}
\newtheorem{corollary}{Corollary}
```

Then state and prove, as usual, a few results that build on one another:

```
\begin{lemma}\label{lem:even}
  The sum of two even integers is even.
\end{lemma}
```

```

\begin{lemma}\label{lem:odd}
  The sum of two odd integers is even.
\end{lemma}
\begin{theorem}\label{thm:parity}
  The sum of two integers of the same parity is even.
\end{theorem}
\begin{proof}
  By Lemmas~\ref{lem:even} and~\ref{lem:odd}; see~\cite{euclid}.
\end{proof}
\begin{corollary}\label{cor:double}
  For every integer  $n$ , the number  $2n$  is even.
\end{corollary}
\begin{proof}
  Apply Theorem~\ref{thm:parity} to  $n$  and  $n$ .
\end{proof}

```

We obtain the following results, exactly as without the package:

Lemma 1. *The sum of two even integers is even.*

Lemma 2. *The sum of two odd integers is even.*

Theorem 1. *The sum of two integers of the same parity is even.*

Proof. By Lemmas 1 and 2; see [1]. □

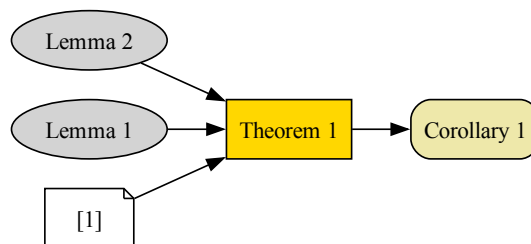
Corollary 1. *For every integer n , the number $2n$ is even.*

Proof. Apply Theorem 1 to n and n . □

In addition, `proofgraph` writes out the dependency graph. With the `autorun` option it is rendered with `Graphviz` and included wherever you call `\proofgraph`:

```
\proofgraph[width=0.7\textwidth]
```

which here produces:



Each cross-reference made inside a proof becomes an arrow *into* the result that proof establishes: the two lemmas point to Theorem 1, which in turn points to Corollary 1. The `\cite` in the proof of Theorem 1 is captured in the same way, thanks to the `cite` option: the external work appears as a distinct node (drawn note-shaped, and labelled here by its printed citation tag, “[1]”, because of `citelabel=tag`) with an arrow into the theorem it helps prove. Moreover, since

this manual loads `hyperref` and `TikZ`, the graph’s nodes are *clickable*: try clicking one to jump to the result it represents (see `\proofgraph` and the `hyperlinks` option).

A handful of commands tune such a graph: `\proofgraphstyle` sets the appearance of each kind of node (the colours here come from it); `\uses` and `\proofgraphedge` record a dependency that no visible `\ref` expresses; `\proofof` attaches a proof to its result; `\proofgraphuntrack` keeps an environment out of the graph; and `\proofgraphignore` and `\proofgraphexclude` drop an unwanted edge or node. The next section puts several of these to work on a real article; Section 3 then documents them, and the package options, in full.

2 A real-world example

The introductory example is deliberately tiny. To show what `proofgraph` produces on a genuine article, the graph in Figure 1 is the one obtained from *Connecting Knowledge Compilation Classes and Width Parameters* [2]. It has 62 numbered results, drawn from five environments (`result`, `theorem`, `proposition`, `corollary` and `lemma`), depends on 19 external works cited inside proofs, and contains 106 edges in all.

The graph was produced from the paper essentially as published. Beyond loading the package, the only additions were the following.

- The `cite` option was switched on, and citation nodes labelled by their printed tag, by loading the package as `\usepackage[cite=true,citelabel=tag]{proofgraph}`. The `\cites` made inside proofs then appear as the note-shaped nodes (“[17]”, “Theorem 4.10, [46]”...) along the left of the graph; a work cited for two different results, or for two of its own theorems, gives one node per `\cite` note, so the 19 works appear as 32 nodes.
- One environment was kept out of the graph with a single `\proofgraphuntrack{observation}` (a lone preliminary observation that nothing else uses).
- The five result kinds were styled, with decreasing prominence, by five `\proofgraphstyle` declarations:

```
\proofgraphstyle{result}{shape=box,style="filled,bold",%
                        fillcolor=orange,peripheries=2}
\proofgraphstyle{theorem}{shape=box,style=filled,%
                        fillcolor=gold,penwidth=2}
\proofgraphstyle{corollary}{shape=box,style="rounded,filled",%
                        fillcolor=palegoldenrod}
\proofgraphstyle{proposition}{shape=box,style="rounded,filled",%
                        fillcolor=lightblue}
\proofgraphstyle{lemma}{shape=ellipse,style=filled,%
                        fillcolor=lightgray}
```

- Eleven `\proofgraphedge` commands, recording 27 edges in total, were added for the dependencies that no `\ref` inside a `proof` expresses: eight “obvious” corollaries stated without a `proof` environment

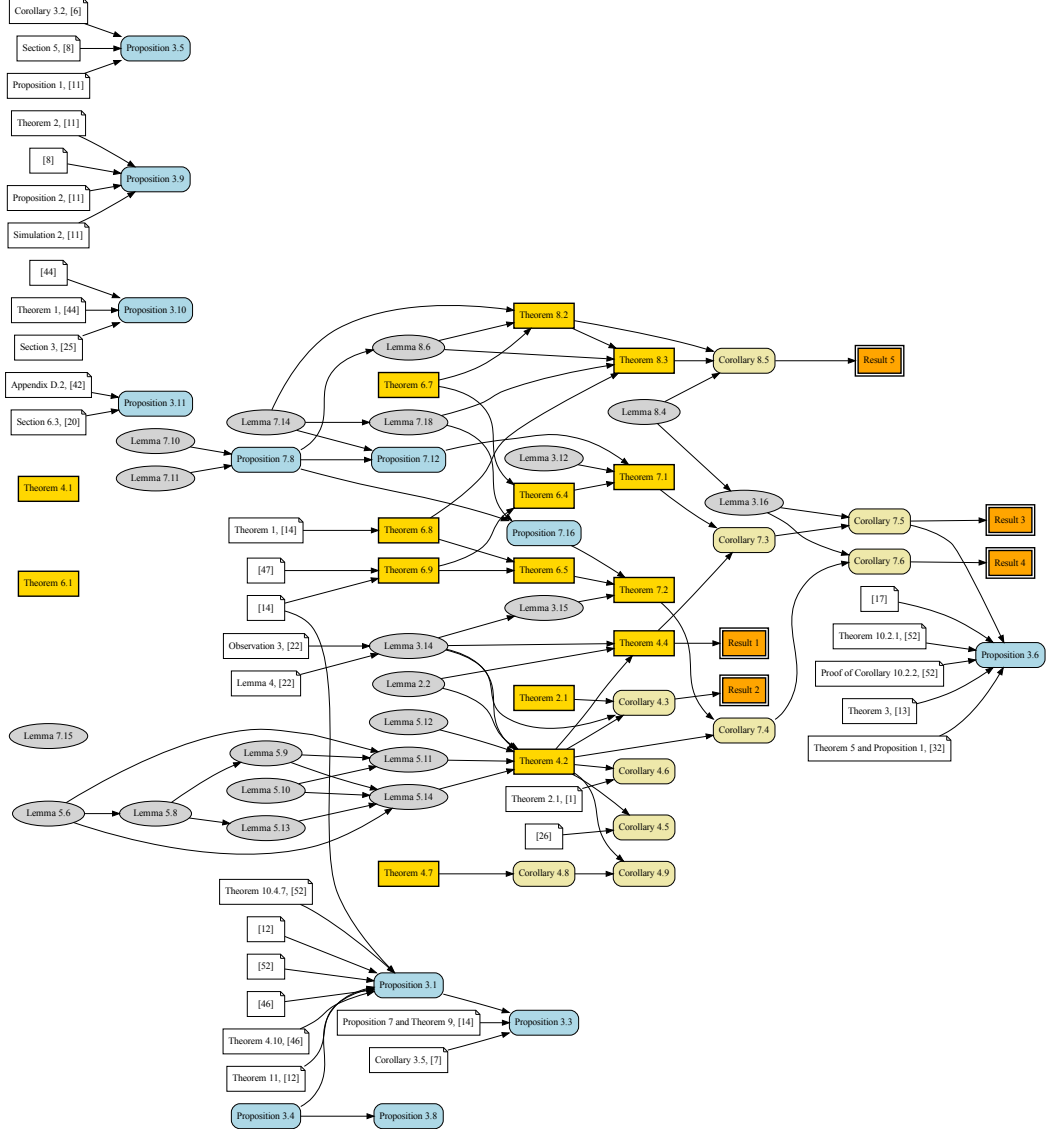


Figure 1: The graph `proofgraph` produces for [2]. The colours and shapes make the structure legible, with the orange double-bordered introduction **results** on the right, fed by the gold theorems, the blue and pale-gold propositions and corollaries, and the grey lemmas.

(`\proofgraphedge{cor:upper_bound}{thm:upper_bound,...}`), and three theorems whose proof is developed across a whole section rather than inside a single `proof`.

Nothing else was annotated: every other edge, and the numbering of all 62 nodes, was inferred automatically from the ordinary cross-references the proofs already contained. In particular, no `\uses`, `\proofof`, `\proofgraphignore` or `\proofgraphexclude` was needed. The five `result` environments that restate the headline results in the introduction (`\begin{result}[(Corollary~\ref{cor:obdd_omega})]...`) are linked to the body results they preview through the `\ref` in their optional title, captured automatically (see the note on reference-carrying titles in Section 3). Two compilation runs (plus `BITEX`, needed once for the `tag` labels) suffice; the `.dot` file is then rendered with `Graphviz`, `dot -Tpdf main.dot -o main.pdf`, as with any other document.

3 Usage

3.1 Default behaviour

Load `proofgraph` in the preamble. It only needs to come *after* whatever provides your theorem environments, that is, after `\usepackage{amsthm}` if you load it yourself (some classes, such as `lipics` or `acmart`, provide theorems for you, in which case there is nothing to load first), and *before* the `\newtheorem` commands that declare your theorem-like environments, so that `proofgraph` sees them as they are created:

```
\usepackage{amsthm}      % or nothing, if your class provides theorems
\usepackage{proofgraph}
\newtheorem{theorem}{Theorem}
\newtheorem{lemma}{Lemma}
```

Then write your document as usual. After two compilation runs, a file `\jobname.dot` is produced. Render it with `Graphviz`:

```
dot -Tpdf myfile.dot -o mygraph.pdf
```

Every *numbered* result of a theorem-like environment you declare becomes a node, and every cross-reference (`\ref`, `\cref`...) inside a proof becomes an edge into the result that proof establishes, with no annotation. The commands captured are `\ref`, `\cref` and `\Cref` (`cleveref`), `\autoref` (`hyperref`), `\eqref` (`amsmath`), `\vref` (`varioref`) and `\zceref` (`zref-clever`, in every form: starred, with options, and with a comma-separated list of labels), each of them whenever it is defined. Purely page-related references (`\pageref`, `\cpageref`, `\zcpageref`) are not captured, as they point at a place rather than at a result; `\uses` records a dependency they would have expressed. Nodes are labelled by the result's formatted name and number ("Theorem 1"), not the environment name. Unnumbered (starred) results are skipped, as they are usually expository or repeated statements; `\proofgraphtrack` draws a chosen one anyway. A cross-reference in the *optional title* of a result is captured the same way: `\begin{theorem}[(generalises~\ref{thm:x})]` in a result labelled $\langle A \rangle$ adds an edge from $\langle A \rangle$ to `thm:x`, handy for restatements (e.g., an introduction that

previews results with `\begin{result}[(Corollary~\ref{cor:y})]`. The *body* of a result is read the same way: a statement that refers to an earlier result, “Under the hypotheses of Theorem 2...”, depends on it just as a proof would. Set `statements=false` to build the graph from proofs alone.

3.2 Recording dependencies by hand

`\uses{<label>}` Inside a proof, records a dependency on the result labelled `<label>` even when no visible reference is made. A comma-separated list is accepted, as in `\uses{lem:a,lem:b}`.

`\proofgraphedge{<A>}{<labels>}` Adds edges recording that result `<A>` depends on the result(s) `<labels>` (a comma-separated list), like a `\uses` but stated outside any proof. Handy for an “obvious” corollary that relies on earlier results without a `proof` environment, as in `\proofgraphedge{cor:x}{thm:y,lem:z}`. It may appear anywhere; all labels are resolved when the graph is written.

`\usescite[<note>]{<keys>}` Inside a proof, the `\uses` of external work: records a dependency on the bibliography entries `<keys>` (a comma-separated list) and draws them as citation nodes, without citing them visibly. The optional `<note>` is the one `\cite` would have taken, as in `\usescite[Thm~3]{knuth}`; it labels the node and, as with a captured `\cite`, tells one result of a work from another.

`\proofgraphciteedge[<note>]{<A>}{<keys>}` The `\proofgraphedge` of citation nodes: records that result `<A>` depends on the cited work(s) `<keys>`, stated outside any proof, as in `\proofgraphciteedge{thm:x}{knuth}`. Useful for a result stated without a proof, which has no place to hold a citation. Like `\proofgraphedge` it may appear anywhere, all labels being resolved when the graph is written.

Both commands create the citation node when the work has none yet, and both work whether or not the `cite` option is on: that option only says whether a plain `\cite` is captured on its own. A work reached both ways, once by `\cite` capture and once by hand, still yields a single node.

`\proofof{<label>}` By default a proof is attached to the most recently started result. A detached proof whose optional title names its result is attached automatically: `\begin{proof}` with `[of Theorem~\ref{thm:x}]` attaches the proof to `thm:x` (for the `amsthm` proof environment, for `proof` defined as a theorem in the Springer classes, and for deferred proofs as with `apxproof`). When none of this applies, `\proofof` inside the proof pins it to the result labelled `<label>`.

3.3 Choosing what appears in the graph

Result environments are detected automatically; these commands adjust which results, and which edges, are kept.

`\proofgraphtrack{<names>}` Forces the listed environment types to be tracked, overriding the default exclusions, which are `definition`, `example`, `remark` and `note`. Use it in the preamble.

`\proofgraphuntrack{<names>}` Excludes further environment types, as in `\proofgraphuntrack{observation}`. Use it in the preamble.

`\proofgraphproofenv{<names>}` Treats the listed environments as proofs, besides `proof`, so that the references they make become dependencies of the result they establish. Use it in the preamble, as in `\proofgraphproofenv{pf}`. It serves a document whose proofs live in an environment of another name, for instance because a proof package had to be renamed to avoid the clash with `amsthm`'s `proof` (as with `pf2`), and equally a `\newtheorem{claimproof}` used for the proofs of claims. Being proofs, the environments named are also removed from the results drawn, even when they are numbered; a `\proofgraphtrack` on the same name overrides that, for an environment to be drawn as a result as well.

An environment taking an optional title is wrapped as `proof` is, so a detached `\begin{pf}[of Theorem~\ref{thm:x}]` attaches to `thm:x`; one declared with `\newtheorem` attaches to the result it follows, or to the one `\proofof` names.

`\proofgraphignore{<from>}{<to>}` Suppresses the edge from `<from>` to `<to>` (e.g., an unwanted forward citation).

`\proofgraphexclude{<label>}` Removes the result labelled `<label>`, together with all its incident edges, from the graph.

`\proofgraphcitecommand{<names>}` Captures further citation commands, given as a comma-separated list of names without their backslash, as in `\proofgraphcitecommand{citeauthor,citeyear}`. Use it in the preamble. Any command taking the usual `<cs>*[<pre>][<post>]{<keys>}` arguments works, including one you define yourself. Naming a command the document does not define is harmless.

It cannot help with the `biblatex` *multicite* commands (`\cites`, `\parencites`, `\textcites`), whose `((<pre>))(<post>)` syntax and repeated key groups `proofgraph` does not parse; cite such works individually, or record them with `\uses`.

3.4 Styling the graph

The attributes are passed verbatim to `Graphviz`; see its attribute reference, <https://graphviz.org/doc/info/attrs.html>, for the node attributes available (and <https://graphviz.org/doc/info/shapes.html> for the shapes).

`\proofgraphstyle{<env>}{<attrs>}` Sets `Graphviz` node attributes for all results of the environment `<env>`, e.g., `shape=ellipse` or `style=filled,fillcolor=gold`.

`\proofgraphstylecite{<attrs>}` Sets the `Graphviz` attributes of the external citation nodes added by the `cite` option (default `shape=note`), as in `\proofgraphstylecite{shape=box,style=dashed,color=gray}`.

3.5 Embedding the rendered graph

`\proofgraph[options]` Includes the rendered graph image at this point, passing the optional *options* to the underlying `\includegraphics` (e.g., `\proofgraph[width=\linewidth]`). This is the only feature that needs `graphicx`, which the package therefore does not load on its own: a document that uses `\proofgraph` must load `graphicx` itself (loading `tikz` for the `hyperlinks` overlay also suffices). With the `autorun` option, `Graphviz` is run at the end of a compilation in which the graph changed (shell-escape required) to produce that image, so `\proofgraph` embeds the graph from the previous run and is up to date after two runs. Without `autorun`, no `Graphviz` is run: `\proofgraph` embeds a pre-existing `\jobname-proofgraph.<format>` file if you have rendered one under that name, and otherwise warns. When `hyperref` and `TikZ` are both loaded (with `hyperlinks` on and the `-Tplain` file from `autorun` present), every node becomes an internal hyperlink to the result it represents: `Graphviz` writes the node positions to that `-Tplain` file and `\proofgraph` overlays a transparent `\hyperref` area on each node (needed because the link annotations `Graphviz` puts in the `.pdf` are discarded by `\includegraphics`).

3.6 Options

selfloops Either `remove` (default), dropping edges from a result to itself, or `keep`, retaining them.

autorun Boolean (default false). Runs `Graphviz` automatically at the end of the run; requires compilation with shell-escape enabled. `Graphviz` is run only when the graph, the engine or the format has changed since the last rendering, so that repeated compilations converge and a build tool such as `latexmk` settles instead of rerunning indefinitely.

engine `Graphviz` layout engine (default `dot`); any `Graphviz` engine works, such as `neato`, `fdp`, `sfdp`, `circo` or `twopi` (see <https://graphviz.org/docs/layouts/>).

format Output format for `autorun` and `\proofgraph` (default `pdf`); any `Graphviz` output format works, such as `png`, `svg` or `ps` (see <https://graphviz.org/docs/outputs/>).

hyperlinks Boolean (default true). Makes the nodes of an embedded graph clickable when `hyperref` and `TikZ` are available (see `\proofgraph`); set it false to embed the graph as a plain image.

direction Either `usedby` (default), orienting an edge from the result that is used to the result that uses it (so an arrow `a->b` reads “a is used by b”), or `uses`, the reverse.

rankdir `Graphviz` graph direction (default `LR`).

cite Boolean (default false). Also captures `\cite` inside proofs as dependencies on external work, drawn as distinct nodes (Section 2), and, unless `statements` is off, in the body of a result too. A `\cite` note is kept:

`\cite[Thm~3]{key}` labels the node “Thm 3, ...”. In the two-optional-argument form of `natbib` and `biblatex`, `\cite[see][Thm~3]{key}`, the post-note likewise labels the node and the pre-note is ignored. Every citation command of L^AT_EX, `natbib` and `biblatex` that cites a work is captured the same way, whenever the document defines it, starred variants included: `\citet`, `\citep`, `\citealt`, `\citealp`, `\parencite`, `\textcite`, `\autocite`, `\footcite`, `\footcitetext`, `\smartcite`, `\supercite`, `\fullcite` and `\footfullcite`, together with the commands printing only a fragment of a citation (`\citeauthor`, `\citefullauthor`, `\citetitle`, `\citeyear`, `\citeyearpar`, `\citedate`, `\citeurl`, `\citenum`) and all their capitalised forms. A work invoked twice in one proof, by `\citeauthor` and by `\citesay`, still yields a single edge. `\proofgraphcitecommand` adds any further command; `\usescite` and `\proofgraphciteedge` record a citation node by hand, with or without this option.

statements Boolean (default true). Captures cross-references, and citations when `\cite` is on, in the *body* of a result as well, not only in its proof and in its optional title: a statement reading “Under the hypotheses of Theorem 2...” or “... the bound of [?, Thm 3]” records that dependency on the result being stated, as a proof making the same reference would. The capture stops at the `\end` of the result, so references in the surrounding text record nothing. Set it false for a graph built from proofs alone, which is the right reading when statements refer to earlier results to contrast with them rather than to rely on them.

citelabel Either `key` (default), labelling citation nodes by their B^BT_EX key, or `tag`, using the printed citation tag instead (“[1]”, “[Knu74]”, “[Abiteboul et al. 1995]”...), which is only known after a first run (so it needs two runs). It supports the standard, `hyperref`-wrapped and `natbib` (numeric and author–year) bibliographies. `biblatex` is supported in part: the tag is recovered for its numeric styles ([1]) and its alphabetic ones ([Knu68]), but not for its author–year styles, whose tag is a formatted name list that `biblatex` exposes as no single field. Whenever no clean tag can be recovered the node keeps the key, which is always a correct, if terser, label.

file Base name of the output file (default `\jobname`).

3.7 Use with `apxproof`

`apxproof` moves proofs to the appendix and typesets them there, so the references a proof makes are only executed at that later point. To attribute them to the right result, `proofgraph` relies on the hook `\apxproofhook`, which `apxproof` fires inside each deferred proof. This hook is available in `apxproof v1.4.1` and later; load the two packages in either order and no further setup is needed.

With an *earlier* `apxproof` the document still compiles cleanly and every result still becomes a node, but *no proof dependencies are captured for the proofs deferred to the appendix*, because `proofgraph` cannot tell which appendix proof belongs to which result (dependencies from any proofs left in the main text are still captured as usual). In that case, add the deferred ones manually with `\uses` (and `\proofof` to pin a proof to its result), or upgrade `apxproof`.

4 Limitations

- Environments declared through `\newtheorem` (or `\spnewtheorem`) *after* the package is loaded are tracked automatically. Environments predefined by the document class *before* that (as in `lipics`, `acmart` or the Springer classes) are also detected, provided their name is one of the common result names `proofgraph` probes at `\begin{document}` (`theorem`, `lemma`, `proposition`, `corollary`...); an environment with an unusual name predefined before the package is loaded must be added with `\proofgraphtrack`.
- Results must be labelled for references to them to be resolved.
- Two compilation runs are required, as for any cross-reference.

References

- [1] Euclid. *Elements*, Book IX. c. 300 BCE.
- [2] A. Amarilli, F. Capelli, M. Monet, and P. Senellart. Connecting knowledge compilation classes and width parameters. *Theory of Computing Systems*, **64**(5):861–914, 2020. doi:10.1007/s00224-019-09930-2.

5 Implementation

The format required is that of October 2020, the first to provide `\NewDocumentCommand` in the kernel; `expl3`, which the package also uses, has been preloaded since the February 2020 release.

```

1 (*package)
2 \NeedsTeXFormat{LaTeX2e}[2020/10/01]
3 \ProvidesPackage{proofgraph}
4 [2026/07/28 v1.0.1-dev Dependency graph of results]

```

5.1 Dependencies

`amsthm` (loaded for its heading hooks) defines the `proof` environment with `\newenvironment`, which aborts with “Command `\proof` already defined” under a class that already provides one (e.g. the Springer `svjour3` `\proof`). When we are about to load `amsthm` ourselves (it is not already loaded), we save the class’s `proof/\endproof` and free them so `amsthm` loads cleanly, then restore the class’s versions afterwards, so its proof styling is kept (and `proofgraph` still hooks that `proof` environment). If `amsthm` is already loaded, `\RequirePackage` is a no-op and `\proof` is left untouched.

```

5 \ifpackageloaded{amsthm}{\}%
6 \ifundefined{proof}{\}%
7 \let\pgph@class@proof\proof \let\pgph@class@endproof\endproof}%
8 \let\proof\relax \let\endproof\relax}
9 \RequirePackage{amsthm}
10 \ifundefined{pgph@class@proof}{\}%
11 \let\proof\pgph@class@proof \let\endproof\pgph@class@endproof}
12 \RequirePackage{etoolbox}
13 \RequirePackage{xstring}
14 \RequirePackage{kvoptions}
15 \RequirePackage{pdftexcmds}

```

`pdftexcmds` provides `\pdf@filemdfivesum`, with which `autorun` recognises an unchanged graph and skips the `Graphviz` run (see `\pgph@maybrender`). On an engine that offers no file checksum the package still works, only without that optimisation.

`graphicx` is needed only to embed a rendered graph with `\proofgraph`; the core `.dot`-writing machinery does not use it. Rather than burden every document with it, we leave it unloaded and check for `\includegraphics` at the point of use (see `\proofgraph`). When the `hyperlinks` overlay is active `tikz` is loaded, which pulls `graphicx` in anyway.

`\pgph@trimsp` An expandable space-trimmer (labels inside a `\cite/\cref/\uses` list arrive with the space after each comma), built on the `expl3` primitive so it is robust to leading spaces and braces.

```

16 \ExplSyntaxOn
17 \cs_new:Npn \pgph@trimsp #1 { \tl_trim_spaces:n {#1} }
18 \ExplSyntaxOff

```

5.2 Options

```

19 \SetupKeyvalOptions{family=pgph,prefix=pgph@}
20 \DeclareStringOption[remove]{selfloops}

```

```

21 \DeclareBoolOption{autorun}
22 \DeclareStringOption[dot]{engine}
23 \DeclareStringOption[pdf]{format}
24 \DeclareStringOption[LR]{rankdir}
25 \DeclareStringOption[usedby]{direction}
26 \DeclareBoolOption{cite}
27 \DeclareStringOption[key]{citelabel}
28 \DeclareBoolOption[true]{statements}
29 \DeclareBoolOption[true]{hyperlinks}
30 \DeclareStringOption{file}
31 \ProcessKeyvalOptions*
32 \ifx\pgph@file\@empty\def\pgph@file{\jobname}\fi

```

5.3 Literal braces

We need catcode-12 braces to write the Graphviz digraph `{ }` block.

```

33 \begingroup
34   \catcode'\ [=1 \catcode'\ ]=2
35   \catcode'\ {=12 \catcode'\ }=12
36   \gdef\pgph@ob[{}]%
37   \gdef\pgph@cb[{}]%
38 \endgroup

```

5.4 Data structures

A global counter gives each result instance a unique numeric id. Nodes and edges are accumulated in list macros and emitted at end of document, so that editing commands (ignore, exclude, self-loops) can be applied to the complete graph regardless of where they appear.

```

39 \newcount\pgph@idcnt
40 \newcount\pgph@citeslotcnt
41 \def\pgph@nodes{}
42 \def\pgph@edges{}
43 \def\pgph@ignores{}
44 \def\pgph@excludes{}
45 \let\pgph@curresult\@empty
46 \let\pgph@curproof\@empty

```

`\pgph@auxmap` The label-to-id map. `\label` inside a result records, both in memory and in the `.aux` file, that the user label maps to the current node id, so references resolve independently of `hyperref`.

```

47 \newcommand\pgph@auxmap[2]{\global\@namedef{pgph@map@#1}{#2}%
48   \ifcsname pgph@rmap@#2\endcsname\else
49     \global\@namedef{pgph@rmap@#2}{#1}\fi}
50 \newcommand\pgph@setmap[2]{%
51   \edef\pgph@tmpid{#2}%
52   \global\expandafter\edef\csname pgph@map@#1\endcsname{\pgph@tmpid}%
53   \ifcsname pgph@rmap@\pgph@tmpid\endcsname\else
54     \global\expandafter\def\csname pgph@rmap@\pgph@tmpid\endcsname{#1}%
55   \fi
56   \protected@write\@auxout{}{\string\pgph@auxmap{#1}{\pgph@tmpid}}%
57 }

```

5.5 Registering result environments

`\pgph@register` A result environment is tracked by installing a begin-hook that records a node. In every declaration command (`\newtheorem`, `\spnewtheorem`) the environment *name* is the first mandatory argument, so registration does not need to parse the remaining (varied) arguments. Each name met is queued as a *candidate* (deduplicated); the begin-hooks themselves are installed only at `\begin{document}`, once the inclusion and exclusion lists are settled, so the customisation commands below may appear anywhere in the preamble.

```

58 \let\pgph@candidates\@empty
59 \newcommand\pgph@register[1]{%
60   \edef\pgph@thisname{#1}%
61   \IfBeginWith\pgph@thisname{axp@}{-}{%
62     \ifcsname pgph@cand@\pgph@thisname\endcsname\else
63       \global\@namedef{pgph@cand@\pgph@thisname}{-}%
64       \ifx\pgph@candidates\@empty
65         \global\let\pgph@candidates\pgph@thisname
66       \else
67         \xdef\pgph@candidates{\pgph@candidates,\pgph@thisname}%
68       \fi
69     \fi
70   }%
71 }
```

We wrap `\newtheorem` (and, for Springer classes, `\spnewtheorem`) to register each newly declared environment, replaying the original command so its own argument parsing is untouched. The starred (unnumbered) forms are *not* auto-registered: a results graph wants numbered statements, and these forms are commonly used for unnumbered repeats of a result (e.g., the appendix copies produced by `apxproof` and `myproofs`), which would otherwise appear as spurious unnumbered duplicate nodes. Use `\proofgraphtrack` to track an unnumbered environment deliberately.

```

72 \let\pgph@orig@newtheorem\newtheorem
73 \renewcommand\newtheorem{\@ifstar{\pgph@nt@star}{\pgph@nt@plain}}
74 \newcommand\pgph@nt@plain[1]{%
75   \pgph@register{#1}\pgph@orig@newtheorem{#1}}
76 \newcommand\pgph@nt@star[1]{\pgph@orig@newtheorem*{#1}}
77 \ifdefined\spnewtheorem
78   \let\pgph@orig@spnewtheorem\spnewtheorem
79   \renewcommand\spnewtheorem{%
80     \@ifstar{\pgph@spnt@star}{\pgph@spnt@plain}}
81   \newcommand\pgph@spnt@plain[1]{%
82     \pgph@register{#1}\pgph@orig@spnewtheorem{#1}}
83   \newcommand\pgph@spnt@star[1]{\pgph@orig@spnewtheorem*{#1}}
84 \fi
```

`\proofgraphtrack`
`\proofgraphuntrack`
`\proofgraphtrack`
`\proofgraphuntrack`

Result environments are detected automatically, but the analysis can be tuned. `\proofgraphtrack{⟨names⟩}` forces the listed environments to be tracked, overriding any exclusion; `\proofgraphuntrack{⟨names⟩}` excludes the listed environment *types*. Expository environments are excluded by default: `definition`, `example`, `remark`, `note`. Both commands belong in the

preamble. `\pgph@defaultenvs` is the list of class-predefined names probed at `\begin{document}`; `\pgph@defaultexclude` seeds the exclusion set.

```

85 \newcommand\pgph@defaultenvs{theorem,lemma,proposition,corollary,%
86   definition,conjecture,claim,fact,remark,example,observation,%
87   notation,problem,question,property,assumption,hypothesis,principle}
88 \newcommand\pgph@defaultexclude{definition,example,remark,note}
89 \newcommand\proofgraphtrack[1]{%
90   \@for\pgph@one:=#1\do{%
91     \edef\pgph@one{\expandafter\pgph@trimsp\expandafter{\pgph@one}}%
92     \csundef{pgph@xtype@\pgph@one}%
93     \global\@namedef{pgph@force@\pgph@one}{}%
94     \expandafter\pgph@register\expandafter{\pgph@one}%
95   }%
96 }
97 \newcommand\proofgraphuntrack[1]{%
98   \@for\pgph@one:=#1\do{%
99     \edef\pgph@one{\expandafter\pgph@trimsp\expandafter{\pgph@one}}%
100    \csundef{pgph@force@\pgph@one}%
101    \global\@namedef{pgph@xtype@\pgph@one}{}%
102  }%
103 }
104 \expandafter\proofgraphuntrack\expandafter{\pgph@defaultexclude}

```

Environments predefined by the document class (acmart, lipics, Springer classes) are declared before the package is loaded, so the wrappers above never see them. At `\begin{document}` we add every common result name that exists to the candidate list, then install a begin-hook for each candidate that is forced or not excluded.

```

105 \newcommand\pgph@dohook[1]{%
106   \ifcsname pgph@hooked@#1\endcsname\else
107     \global\@namedef{pgph@hooked@#1}{}%
108     \AtBeginEnvironment{#1}{\pgph@beginresult{#1}}%
109   \fi
110 }
111 \newcommand\pgph@maybehook[1]{%
112   \ifcsname pgph@force@#1\endcsname
113     \pgph@dohook{#1}%
114   \else
115     \ifcsname pgph@xtype@#1\endcsname\else\pgph@dohook{#1}\fi
116   \fi
117 }
118 \newcommand\pgph@resolveregistration[1]{%
119   \@for\pgph@one:=#1\do{%
120     \ifcsname\pgph@one\endcsname%
121       \expandafter\pgph@register\expandafter{\pgph@one}\fi%
122   \ifx\pgph@candidates\@empty\else
123     \@for\pgph@one:=\pgph@candidates\do{%
124       \expandafter\pgph@maybehook\expandafter{\pgph@one}}%
125   \fi
126 }
127 \AtBeginDocument{%
128   \expandafter\pgph@resolveregistration\expandafter{\pgph@defaultenvs}

```

`\pgph@beginresult` Start a result: allocate an id and record the node. The node is keyed and styled by `\pgph@hooknames`
`\pgph@recordresult`

the environment name (third argument, also a fallback label), but displayed by the result's *formatted* name ("Theorem", "Lemma"), captured from the heading. We get that name by locally wrapping the heading macros, whose first two arguments are the formatted name and the number. Which macro carries the *optional note* (where a restatement's cross-reference lives) depends on the setup: `amsthm`, which `proofgraph` always loads, routes every heading through `\@begintheorem`, defined as `#1#2[#3]` with the note as the bracketed `#3` (and leaves `\@opargbegintheorem` `\relax`); the L^AT_EX kernel instead uses a two-argument `\@begintheorem` with a separate three-argument `\@opargbegintheorem` for the note; the Springer classes use `\@spbtheorem`/`\@spopargbegintheorem`. We detect the `amsthm` form (`\@opargbegintheorem` is `\relax`) and read the bracketed note there, otherwise we hook `\@opargbegintheorem`; either way the note goes to `\pgph@scanrefs`. These redefinitions are confined to the current environment group, so they revert at `\end`. We also locally redefine `\label` to populate the map.

Nodes are accumulated in *reverse* order of appearance (`\pgph@prependnode`). With `rankdir=LR`, Graphviz stacks disconnected components vertically in the order they are read, from the bottom up; emitting the results last-to-first therefore lays them out top-to-bottom in document order (first result on top), while connected results stay grouped. This is a layout tendency, not a guarantee, but needs no `pack` trickery.

Unless `statements` is switched off, the *body* of the result is treated like the body of a proof of that result (`\pgph@beginstatement`), so that a cross-reference or a citation made in the statement itself becomes a dependency. Those assignments are local to the environment group, as the ones above are, so they revert at `\end`.

```

129 \newcommand\pgph@prependnode[1]{%
130   \xdef\pgph@nodes{\unexpanded{#1}\unexpanded\expandafter{\pgph@nodes}}%
131 }
132 \newcommand\pgph@beginresult[1]{%
133   \global\advance\pgph@idcnt\@ne
134   \xdef\pgph@curresult{\the\pgph@idcnt}%
135   \protected@edef\pgph@tmp{%
136     \noexpand\pgph@node{\pgph@curresult}{#1}{#1}}%
137   \expandafter\pgph@prependnode\expandafter{\pgph@tmp}%
138   \pgph@hooknames
139   \ifx\label\pgph@maplabel\else\let\pgph@savedlabel\label\fi
140   \let\label\pgph@maplabel
141   \ifpgph@statements\pgph@beginstatement\fi
142 }
143 \newcommand\pgph@hooknames{%
144   \ifdefined\@begintheorem
145     \let\pgph@orig@bt\@begintheorem
146     \ifx\@opargbegintheorem\relax
147       \def\@begintheorem##1##2[#3]{%
148         \pgph@recordresult{##1}{##2}\pgph@scanrefs{##3}%
149         \pgph@orig@bt{##1}{##2}[##3]}%
150     \else
151       \def\@begintheorem##1##2{%
152         \pgph@recordresult{##1}{##2}\pgph@orig@bt{##1}{##2}}%
153     \fi
154   \fi
155   \ifdefined\@opargbegintheorem\ifx\@opargbegintheorem\relax\else
156     \let\pgph@orig@obt\@opargbegintheorem

```

```

157 \def\@opargbegintheorem##1##2##3{%
158 \pgph@recordresult{##1}-{##2}\pgph@scanrefs{##3}%
159 \pgph@orig@obt{##1}-{##2}-{##3}}%
160 \fi\fi
161 \ifdefined\@spbegintheorem
162 \let\pgph@orig@spbt\@spbegintheorem
163 \def\@spbegintheorem##1##2##3##4{%
164 \pgph@recordresult{##1}-{##2}\pgph@orig@spbt{##1}-{##2}-{##3}-{##4}}%
165 \fi
166 \ifdefined\@spopargbegintheorem
167 \let\pgph@orig@spobt\@spopargbegintheorem
168 \def\@spopargbegintheorem##1##2##3##4##5{%
169 \pgph@recordresult{##1}-{##2}\pgph@scanrefs{##3}%
170 \pgph@orig@spobt{##1}-{##2}-{##3}-{##4}-{##5}}%
171 \fi
172 }

```

`\pgph@scanrefs` A result whose optional title carries a cross-reference, e.g., a `result` whose title is `[(Corollary~\ref{cor:x})]`, is understood to restate or build on that result: we record an edge from the current result to each label referenced in the title. We capture these by typesetting the title once into a discarded box with the capturing commands installed and `\pgph@curproof` pointing at the current result. Everything happens inside a group and `\pgph@curproof` is only *read* while the box is built, so a local assignment suffices and nothing outside the title is affected; a global one would escape the enclosing environment, which under `statements` (where `\pgph@curproof` is already set on entering the result) would leave every subsequent reference attributed to that result.

```

173 \newcommand\pgph@scanrefs[1]{%
174 \ifx\pgph@curresult\@empty\else
175 \begingroup
176 \let\pgph@curproof\pgph@curresult
177 \pgph@installcapture
178 \setbox\z@\hbox{#1}%
179 \endgroup
180 \fi
181 }

```

The formatted name is the first argument of every heading macro (`\@begintheorem/\@opargbegintheorem` for `amsthm`/`\newtheorem`, `\@spbegintheorem/\@spopargbegintheorem` for Springer's `\spnewtheorem`); the second argument is the number. We use that second argument only to tell *whether* the result is numbered (it is empty for the starred, unnumbered forms): an unnumbered result records no number, so it is dropped at emission unless tracked explicitly. The number *value*, when present, is read from `\@currentlabel` rather than from that argument, which some classes pollute (e.g., `lipics` injects `\advance\par@deathcycles\@ne`). `\@currentlabel`, set by the result's `\refstepcounter` just before the heading, is the clean number and needs no `\label`. Values are reduced to plain strings now, with fragile wrappers neutralized (notably `apxproof`'s forward-link). The first heading wins (guarded on the name), ignoring nested headings. The emptiness of the number argument is tested without expanding it, so a polluted (but non-empty) number does not trip the test.

```

182 \newcommand\pgph@recordresult[2]{%
183   \ifcsname pgph@name@\pgph@curresult\endcsname\else
184     \begingroup
185       \let\protect\@empty
186       \def\axp@forward@link##1##2{##2}%
187       \edef\pgph@tmpname{#1}%
188       \global\expandafter\let\csname pgph@name@\pgph@curresult\endcsname
189         \pgph@tmpname
190       \global\@namedef{pgph@seen@\pgph@curresult}{}%
191       \def\pgph@tmpn{#2}%
192       \ifx\pgph@tmpn\@empty\else
193         \edef\pgph@tmpnum{\@currentlabel}%
194         \global\expandafter\let\csname pgph@num@\pgph@curresult%
195           \endcsname\pgph@tmpnum
196       \fi
197     \endgroup
198   \fi
199 }

```

`\pgph@maplabel` records the label-to-node mapping. As a fallback for classes whose headings pass through none of the hooked heading macros (so no heading was `seen`), it also captures the number from `\@currentlabel`. We do *not* do this once a heading was seen: there, an absent number means the result is genuinely unnumbered (a starred form), and `\@currentlabel` could be a stale value from an earlier result. The value is expanded to a plain string *now*, inside a group where fragile wrappers are neutralized, so that nothing dangerous survives to the emission pass: in particular, under `apxproof` forward-linking, `\@currentlabel` holds `\axp@forward@link{<target>}{<number>}`, whose hyper-link machinery must never be expanded in a non-typesetting context (it would run away). Reducing it to its number argument keeps it harmless.

`\pgph@maplabel`

```

200 \newcommand\pgph@maplabel[1]{%
201   \pgph@setmap{#1}{\pgph@curresult}%
202   \ifcsname pgph@num@\pgph@curresult\endcsname\else
203     \ifcsname pgph@seen@\pgph@curresult\endcsname\else
204       \begingroup
205         \let\protect\@empty
206         \def\axp@forward@link##1##2{##2}%
207         \edef\pgph@tmpnum{\@currentlabel}%
208         \global\expandafter\let\csname pgph@num@\pgph@curresult%
209           \endcsname\pgph@tmpnum
210       \endgroup
211     \fi
212   \fi
213   \pgph@savetlabel{#1}%
214 }

```

5.6 Hooking proofs

Inside a proof we redefine the reference commands to capture edges from the proved result.

```

215 \newcommand\pgph@adddedge[2]{%

```

```

216 \protected@edef\pgph@tmpedge{\noexpand\pgph@edge{#1}{#2}}%
217 \expandafter\g@addto@macro\expandafter\pgph@edges%
218 \expandafter{\pgph@tmpedge}%
219 }
220 \newcommand\pgph@recordref[1]{%
221 \ifx\pgph@curproof\empty\else
222 \@for\pgph@one:=#1\do{%
223 \edef\pgph@k{\expandafter\pgph@trimsp\expandafter{\pgph@one}}%
224 \pgph@addedge{\pgph@curproof}{\pgph@k}}%
225 \fi
226 }

```

`\pgph@beginproof` Attach the proof to the current result and install the capturing versions of the reference commands (only those that exist).

The capturing commands are `\protected` so that, when a reference appears in a moving argument inside a proof (e.g., a `\caption`), they do not expand into the `.aux/.lof` and leak internal tokens.

```

227 \protected\def\pgph@cap@ref#1{\pgph@recordref{#1}\pgph@saved@ref{#1}}
228 \protected\def\pgph@cap@cref#1{\pgph@recordref{#1}\pgph@saved@cref{#1}}
229 \protected\def\pgph@cap@Cref#1{\pgph@recordref{#1}\pgph@saved@Cref{#1}}
230 \protected\def\pgph@cap@autoref#1{%
231 \pgph@recordref{#1}\pgph@saved@autoref{#1}}
232 \protected\def\pgph@cap@eqref#1{%
233 \pgph@recordref{#1}\pgph@saved@eqref{#1}}
234 \protected\def\pgph@cap@vref#1{\pgph@recordref{#1}\pgph@saved@vref{#1}}

```

`zref-clever`'s `\zcref` is the one captured command that is not simply `\langle cs \rangle \langle labels \rangle`: it is `\zcref*[\langle options \rangle]{\langle labels \rangle}`, and both the star and the options have to be handed on unchanged. Its mandatory argument is a comma-separated list, which `\pgph@recordref` already splits.

```

235 \NewDocumentCommand\pgph@cap@zcref{s0}{m}{%
236 \pgph@recordref{#3}%
237 \IfBooleanTF{#1}{\pgph@saved@zcref*{#2}{#3}}{\pgph@saved@zcref{#2}{#3}}%
238 }
239 \newcommand\pgph@install[1]{%
240 \ifcsname #1\endcsname
241 \expandafter\ifx\csname #1%
242 \expandafter\endcsname\csname pgph@cap@#1\endcsname
243 \else
244 \expandafter\let\csname pgph@saved@#1%
245 \expandafter\endcsname\csname #1\endcsname
246 \expandafter\let\csname #1%
247 \expandafter\endcsname\csname pgph@cap@#1\endcsname
248 \fi
249 \fi
250 }

```

The set of captured commands is needed in three places (a proof, the optional title of a result, and an `apxproof` deferred proof), so it is named once here: adding a command to the list must not be a matter of remembering three call sites.

```

251 \newcommand\pgph@installrefs{%
252 \pgph@install{ref}\pgph@install{cref}\pgph@install{Cref}%
253 \pgph@install{autoref}\pgph@install{eqref}\pgph@install{vref}%

```

```

254 \pgph@install{zceref}%
255 }

```

The cross-reference commands and, when the `cite` option is on, the citation commands are always installed together, so the pair has a name of its own: every place that opens a capturing context (a proof, the optional title of a result, and, with `statements`, the body of a result) installs exactly the same set.

```

256 \newcommand\pgph@installcapture{%
257   \pgph@installrefs
258   \ifpgph@cite\pgph@installcite\fi
259 }
260 \newcommand\pgph@beginproof{%
261   \let\pgph@curproof\pgph@curresult
262   \pgph@installcapture
263   \pgph@hookproofhead
264 }

```

`\pgph@beginstatement` Under the `statements` option the body of a result captures like the body of its proof. The proof-heading hook is deliberately *not* installed here: it turns a cross-reference in an optional title into a `\proofof`, which is right for a proof titled “of Theorem 1” but not for a result titled “(restates Theorem 1)”, where the reference is an ordinary dependency.

```

265 \newcommand\pgph@beginstatement{%
266   \let\pgph@curproof\pgph@curresult
267   \pgph@installcapture
268 }

```

When the `proof` environment is itself a theorem-like environment (as with Springer’s `\spnewtheorem*{proof}`), its optional title is set by `\@opargbegintheorem/\@spopargbegintheorem`. We wrap these for the duration of the proof so that a title such as `[of Theorem~\ref{thm:x}]` sets `\proofof{thm:x}`, attributing the proof to its result. As in `\pgph@hooknames`, a `\relax \@opargbegintheorem` is left alone: that is the `amsthm` setup, where the macro is never called, and giving it a meaning would make `\pgph@hooknames` read a result nested in the proof as having the kernel’s two-argument heading rather than `amsthm`’s.

```

269 \newcommand\pgph@hookproofhead{%
270   \ifdefined\@opargbegintheorem\ifx\@opargbegintheorem\relax\else
271     \let\pgph@orig@obtp\@opargbegintheorem
272     \def\@opargbegintheorem##1##2##3{%
273       \pgph@scanproofarg{##3}\pgph@orig@obtp{##1}{##2}{##3}}%
274   \fi\fi
275 \ifdefined\@spopargbegintheorem
276   \let\pgph@orig@spobtp\@spopargbegintheorem
277   \def\@spopargbegintheorem##1##2##3##4##5{%
278     \pgph@scanproofarg{##3}%
279     \pgph@orig@spobtp{##1}{##2}{##3}{##4}{##5}}%
280 \fi
281 \ifdefined\@Opargbegintheorem
282   \let\pgph@orig@Obtp\@Opargbegintheorem
283   \def\@Opargbegintheorem##1##2##3##4{%
284     \pgph@scanproofarg{##2}\pgph@orig@Obtp{##1}{##2}{##3}{##4}}%
285 \fi

```

286 }

`\pgph@scanproofarg` A proof whose optional title names its result (a detached proof) has its result given by that title, e.g., `\thm:x` for an optional argument [`of Theorem~\ref{thm:x}`], rather than by the most recently opened result. We capture this by wrapping the `\proof` environment so that, when an optional title is given, the cross-reference it contains is turned into a `\proofof` (the title is typeset once into a discarded box with `\ref` and friends redefined). Without an optional title the original environment is used unchanged.

```
287 \NewDocumentCommand\pgph@zcproofof{s0{m}}{\proofof{#3}}
288 \newcommand\pgph@scanproofarg[1]{%
289   \begingroup
290     \let\protect\@empty
291     \def\ref##1{\proofof{##1}}\def\cref##1{\proofof{##1}}%
292     \def\Cref##1{\proofof{##1}}\def\autoref##1{\proofof{##1}}%
293     \let\zcref\pgph@zcproofof
294     \setbox\z@\hbox{#1}%
295   \endgroup
296 }
```

To capture the optional title of a detached `\amsthm` proof (an optional [`of Theorem~\ref{thm:x}`]) we redefine the outer `\proof` so that, when a title is given, it is scanned and then handed to the *saved* outer macro, which parses it as it always did. Delegating to the saved outer macro rather than to the inner one means we need to know neither the name of the inner macro nor the environment's default title, so the same wrapper serves any proof environment, whatever its default. No recursion occurs: the saved meaning is captured before the redefinition.

We must wrap *only* a genuine optional-argument environment: a package that captures the proof body verbatim (`\apxproof`, `\myproofs`) redefines `\proof` with no optional argument, and wrapping it would break that capture. We recognise the optional-argument form by the `\@protected@testopt` in its definition. When the proof environment is instead a theorem-like environment (`\spnewtheorem*{proof}`, or a `\newtheorem` of one's own) its title is already handled by `\pgph@hookproofhead`, so there we just install the begin-hook.

```
297 \newcommand\pgph@wrapproofone[1]{%
298   \@ifundefined{#1}{}{%
299     \edef\pgph@proofmeaning{\expandafter\meaning\csname #1\endcsname}%
300     \edef\pgph@testoptstr{\detokenize{\@protected@testopt}}%
301     \IfSubStr\pgph@proofmeaning\pgph@testoptstr
302       {\pgph@redefproof{#1}}%
303       {\AtBeginEnvironment{#1}{\pgph@beginproof}}%
304   }%
305 }
306 \newcommand\pgph@redefproof[1]{%
307   \expandafter\let\csname pgph@savenv@#1\expandafter\endcsname
308     \csname #1\endcsname
309   \expandafter\def\csname #1\endcsname{\pgph@beginproof
310     \@ifnextchar[{\pgph@proofopt{#1}}%
311       {\csname pgph@savenv@#1\endcsname}}%
312 }
313 \def\pgph@proofopt#1[#2]{%
314   \pgph@scanproofarg{#2}\csname pgph@savenv@#1\endcsname[#2]}
```

`\proofgraphproofenv`

`\proofgraphproofenv` Which environments are proofs is a list, `proof` to begin with;
`\pgph@proofenvs` `\proofgraphproofenv{<names>}` adds to it, for a document whose proofs live in an environment of another name (pf2's structured proofs under a name that avoids the clash with `amsthm`, a `\newtheorem{claimproof}` for the proofs of claims...). A proof is not a result, so the names are excluded from result tracking as well; `\proofgraphtrack` still overrides that for an environment one wants drawn as both.

```

315 \newcommand\pgph@proofenvs{proof}
316 \newcommand\proofgraphproofenv[1]{%
317   \g@addto@macro\pgph@proofenvs{,#1}%
318   \proofgraphtrack{#1}%
319 }

```

Packages such as `apxproof` capture the body of a `proof` verbatim (to defer it to the appendix) and replay it there. Patching the `proof` environment then corrupts that capture, and the references are only executed at replay time anyway. So we wrap `proof` only when no such package is active; otherwise the deferred-proof capture (below) takes over. The exemption is for `proof` alone: an environment added with `\proofgraphproofenv` is not one `apxproof` defers, so it is wrapped as usual. This happens at `\begin{document}`, once we know what is loaded and what the preamble has added to the list.

```

320 \newif\ifpgph@apxproof
321 \newcommand\pgph@wrapproofs{%
322   \@for\pgph@one:=\pgph@proofenvs\do{%
323     \edef\pgph@pe{\expandafter\pgph@trimsp\expandafter{\pgph@one}}%
324     \ifboolexpr{ bool {pgph@apxproof} and test {\ifdefstring{\pgph@pe}{proof}} }{%
325       {}{\expandafter\pgph@wrapproofone\expandafter{\pgph@pe}}%
326     }%
327   }
328 \AtBeginDocument{%
329   \@ifpackageloaded{apxproof}{\pgph@apxprooftrue\pgph@apxinit}{}%
330   \pgph@wrapproofs
331 }

```

`\pgph@apxinit` Under `apxproof`, deferred proofs are typeset in the appendix, where their references
`\pgph@apxproofcapture` finally execute. `apxproof` fires `\apxproofhook` inside each such proof, passing the `axp@r<n>` label of the proved theorem, which (because `apxproof` also attaches that label to the theorem in the main text) is already in our map. We resolve it to the source node and install the reference-capturing commands for the duration of the proof.

```

332 \newcommand\pgph@apxinit{%
333   \def\apxproofhook##1{\pgph@apxproofcapture{##1}}
334 \newcommand\pgph@apxproofcapture[1]{%
335   \edef\pgph@curproof{\pgph@resolve{#1}}%
336   \ifx\pgph@curproof\@empty\else
337     \pgph@installcapture
338   \fi
339 }

```

`\pgph@hookcite` Optional citation capture, applied to `\cite` and, when they are defined (`natbib`, `biblatex`), to `\citet` and `\citep` as well, all sharing one capture chain parame-

terized by the command name: external references become `cite:-`-prefixed target labels, drawn later as distinct nodes. The node identity is the pair (key, note): a `\cite[⟨note⟩]{⟨key⟩}` inside a proof is distinguished from a `\cite` of the same key with a *different* note, so that citing two different results of the same work (say two theorems of one book) yields two nodes rather than one mislabelled with whichever note was seen first. Each pair is assigned a numeric *slot* (`\pgph@citeslot`); the edge target is `cite:⟨slot⟩`, and the slot remembers the key (for the tag and the bibliography hyperlink) and the note (for the label). The note is reduced to a plain string at capture time (fragile wrappers neutralized, `~` normalised to a space), which also serves as the signature so `[Thm 1]` and `[Thm~1]` coincide. We peek for the optional argument with `\@ifnextchar` rather than declaring one, so that a note-less `\cite{⟨key⟩}` is passed on *as written*: re-emitting it as `\cite[]{⟨key⟩}` would make the typeset citation grow a spurious empty note (“[1,]”). A second optional argument is peeked for in the same way, to support the two-argument form `\cite[⟨pre-note⟩][⟨note⟩]{⟨key⟩}` of `natbib` and `biblatex`; there the *second* argument is the note recorded for the node (matching the one-argument semantics, where the lone argument is the post-note), while the pre-note (“see”, “e.g.”) qualifies the act of citing rather than identifying the cited result, and is only replayed to the saved `\cite`. Without this peek, the mandatory-argument scan would grab the bare token `[` as the key and typeset the rest of the citation as text, giving confusing errors (“Missing \$ inserted”) deep in the proof body. A star (`\citet*`, `\citep*`: full author list) is likewise peeked for and replayed; it affects rendering only, not the recorded (key, note) pair. The replay head (saved command plus optional star) is frozen in `\pgph@replay` by the star-peek before the arguments are scanned. As in `\pgph@install`, installation is skipped (per command) when the command is already the capture macro; otherwise a proof nested inside another proof would save the capture as the saved command, and the first citation in the inner proof would recurse forever.

```

340 \newcommand\pgph@gxdefcs[2]{%
341   \global\expandafter\edef\csname#1\endcsname#{2}}
342 \newcommand\pgph@cap@citegen[1]{%
343   \def\pgph@citecmd{#1}%
344   \@ifstar{\pgph@cap@cite@x*}{\pgph@cap@cite@x}}
345 \def\pgph@cap@cite@x#1{%
346   \edef\pgph@replay{%
347     \expandafter\noexpand\csname pgph@saved@\pgph@citecmd\endcsname#1}%
348   \@ifnextchar[\pgph@cap@cite@opt\pgph@cap@cite@noopt}
349 \def\pgph@cap@cite@opt[#1]{%
350   \@ifnextchar[\pgph@cap@cite@optii{#1}]{\pgph@cap@cite@opti{#1}}}
351 \def\pgph@cap@cite@opti#1#2{%
352   \pgph@cap@citerec{#1}{#2}\pgph@replay[#1]{#2}}
353 \def\pgph@cap@cite@optii#1[#2]#3{%
354   \pgph@cap@citerec{#2}{#3}\pgph@replay[#1][#2]{#3}}
355 \def\pgph@cap@cite@noopt#1{\pgph@cap@citerec{#1}\pgph@replay{#1}}
356 \newcommand\pgph@cap@citerec[2]{%
357   \ifx\pgph@curproof\@empty\else
358     \@for\pgph@one:=#2\do{%
359       \edef\pgph@k{\expandafter\pgph@trimsp\expandafter{\pgph@one}}%
360       \pgph@citeslot{\pgph@k}{#1}%
361       \pgph@addedge{\pgph@curproof}{cite:\pgph@curslot}}%
362   \fi

```

```

363 }
364 \newcommand\pgph@citeslot[2]{%
365   \begingroup
366     \pgph@citesanitize
367     \xdef\pgph@gnotestr{#2}%
368   \endgroup
369   \edef\pgph@sig{#1@\detokenize\expandafter{\pgph@gnotestr}}%
370   \ifcsname pgph@cslot@\pgph@sig\endcsname
371     \edef\pgph@curslot{\csname pgph@cslot@\pgph@sig\endcsname}%
372   \else
373     \global\advance\pgph@citeslotcnt\@ne
374     \edef\pgph@curslot{\the\pgph@citeslotcnt}%
375     \pgph@gxdefcs{pgph@cslot@\pgph@sig}{\pgph@curslot}%
376     \pgph@gxdefcs{pgph@cskey@\pgph@curslot}{#1}%
377     \ifx\pgph@gnotestr\@empty\else
378       \pgph@gxdefcs{pgph@citenote@\pgph@curslot}{\pgph@gnotestr}%
379     \fi
380   \fi
381 }

```

`\pgph@citecmds` Every citation command with the signature `\cs*[\langle pre \rangle][\langle post \rangle] \{ \langle keys \rangle \}` is captured by the same `\pgph@cap@citegen`, so which commands are captured `\proofgraphcitecommand` is a list rather than code. The default list holds the citation commands of L^AT_EX, natbib and biblatex that cite a work; `\proofgraphcitecommand` adds more. Naming a command that the document never defines costs nothing, as `\pgph@installciteone` installs only over an existing command, so the list may name every spelling a citation package might provide.

The commands printing only a *fragment* of a citation (`\citeauthor`, `\citeyear`...) are in the list as well. They commonly accompany a real citation of the same work, but a repetition is harmless: a (key, note) pair already seen reuses its slot, and `\pgph@writeedge` draws an edge once, so the second mention of a work adds nothing to the graph. Capturing them means that a proof invoking a work by author or by year alone, which does happen, is not silently left without its edge.

The biblatex *multicite* commands (`\cites`, `\parencites`, `\textcites`) cannot be captured at all: their `(\langle pre \rangle)(\langle post \rangle)` syntax and repeated key groups are a different grammar, which `\pgph@cap@citegen` does not parse.

```

382 \newcommand\pgph@citecmds{%
383   cite,Cite,citet,Citet,citep,Citep,%
384   citealt,Citealt,citealp,Citealp,%
385   parencite,Parencite,textcite,Textcite,autocite,Autocite,%
386   footcite,Footcite,footcitetext,smartcite,Smartcite,supercite,%
387   fullcite,footfullcite,%
388   citeauthor,Citeauthor,citefullauthor,citetitle,Citetitle,%
389   citeyear,Citeyear,citeyearpar,citedate,Citedate,citeurl,citenum}
390 \newcommand\proofgraphcitecommand[1]{\g@addto@macro\pgph@citecmds{,#1}}
391 \newcommand\pgph@installcite{%
392   \@for\pgph@conename:=\pgph@citecmds\do{%
393     \edef\pgph@cone{\expandafter\pgph@trimsp\expandafter{\pgph@conename}}%
394     \expandafter\pgph@installciteone\expandafter{\pgph@cone}}%
395 }

```

The capture macro of a command is built on first use rather than declared: it is the

same `\pgph@cap@citegen` call for all of them, and building it only for commands the document actually defines keeps the unused names of the list from costing anything. It must be `\gdef`, the installation happening inside the group of a proof.

```

396 \newcommand\pgph@installciteone[1]{%
397   \ifcsname #1\endcsname
398     \ifcsname pgph@cap@#1\endcsname\else
399       \expandafter\gdef\csname pgph@cap@#1\endcsname{\pgph@cap@citegen{#1}}%
400     \fi
401   \expandafter\ifx\csname #1\endcsname\expandafter\endcsname
402     \csname pgph@cap@#1\endcsname
403   \else
404     \expandafter\let\csname pgph@saved@#1\endcsname\expandafter\endcsname
405     \csname #1\endcsname
406     \expandafter\let\csname #1\endcsname\expandafter\endcsname
407     \csname pgph@cap@#1\endcsname
408   \fi
409 \fi
410 }
```

The `biblatex` tag recorder is needed only when tags are actually wanted, and is registered at the end of the preamble, once `biblatex` has certainly been loaded (it is a preamble-only package) but while `\AtEveryCitekey` still accepts additions. It is not conditioned on the `cite` option, which governs the automatic capture of `\cite` only: `\usescite` and `\proofgraphciteedge` draw citation nodes without it, and those nodes want their tag just the same.

```

411 \AtEndPreamble{%
412   \ifdefstring{\pgph@citelabel}{tag}%
413     {\ifpackageloaded{biblatex}{\AtEveryCitekey{\pgph@blxrecord}}{}}{}}%
414 }
```

5.7 User commands

```

415 \newcommand\uses[1]{\pgph@recordref{#1}}
416 \newcommand\proofof[1]{%
417   \ifcsname pgph@map@#1\endcsname
418     \xdef\pgph@curproof{\csname pgph@map@#1\endcsname}%
419   \fi
420 }
421 \newcommand\proofgraphedge[2]{%
422   \g@addto@macro\pgph@edges{\pgph@labeledge{#1}{#2}}%
423 }
424 \newcommand\pgph@labeledge[2]{%
425   \edef\pgph@ef{\pgph@resolve{#1}}%
426   \ifx\pgph@ef\@empty\else
427     \@for\pgph@one:=#2\do{%
428       \edef\pgph@k{\expandafter\pgph@trimsp\expandafter{\pgph@one}}%
429       \pgph@plainedge{\pgph@ef}{\pgph@k}}%
430   \fi
431 }
```

`\usescite`
`\proofgraphciteedge` The two manual counterparts of the automatic `\cite` capture, for a dependency
`\proofgraphciteedge` on external work that no citation in a proof expresses: `\usescite`, the `\uses`
`\pgph@citelabeledge`

of citation nodes, records one from inside a proof, and `\proofgraphciteedge`, the `\proofgraphedge` of citation nodes, states one from anywhere. Both create the citation node if the work has no node yet, take a comma-separated list of BibTeX keys and accept as an optional argument the note that would have been the optional argument of `\cite` ([Thm~3]), which labels the node and, as in the automatic capture, distinguishes one result of a work from another.

Being explicit, they do not depend on the `cite` option, which only says whether a plain `\cite` is captured on its own. `\usescite` shares the recording path of the captured citation commands, so a work reached both ways still yields a single node.

```
432 \NewDocumentCommand\usescite{0{m}}{\pgph@cap@citerec{#1}{#2}}
433 \NewDocumentCommand\proofgraphciteedge{0{mm}}{%
434   \g@addto@macro\pgph@edges{\pgph@citelabeledge{#1}{#2}{#3}}%
435 }
```

Resolution is deferred to emission, as for `\proofgraphedge`, so the result may be labelled anywhere in the document; the slot is therefore allocated then, which is harmless as slot numbers only have to be distinct. The edge is assembled by `\edef` rather than passed as macros, because `\pgph@citeedge` takes the `cite:<slot>` target apart with `xstring`, which reads its argument literally.

```
436 \newcommand\pgph@citelabeledge[3]{%
437   \edef\pgph@ef{\pgph@resolve{#2}}%
438   \ifx\pgph@ef\@empty\else
439     \@for\pgph@one:=#3\do{%
440       \edef\pgph@k{\expandafter\pgph@trimsp\expandafter{\pgph@one}}%
441       \pgph@citeslot{\pgph@k}{#1}%
442       \edef\pgph@tmpce{%
443         \noexpand\pgph@citeedge{\pgph@ef}{cite:\pgph@curslot}}%
444       \pgph@tmpce}%
445   \fi
446 }

447 \newcommand\proofgraphignore[2]{%
448   \g@addto@macro\pgph@ignores{\pgph@ignorerule{#1}{#2}}%
449 }

450 \newcommand\proofgraphexclude[1]{%
451   \g@addto@macro\pgph@excludes{\pgph@excluderule{#1}}%
452 }

453 \newcommand\proofgraphstyle[2]{\global\@namedef{pgph@style@#1}{#2}}
```

External citation nodes (option `cite`) share one style, held in `\pgph@citestyle` and overridable with `\proofgraphstylecite`; it defaults to `shape=note`.

```
454 \newcommand\pgph@citestyle{shape=note}
455 \newcommand\proofgraphstylecite[1]{\renewcommand\pgph@citestyle{#1}}
```

5.8 Resolution helpers

These run at end of document, after the `.aux` has populated the map.

```
456 \newcommand\pgph@resolve[1]{%
457   \ifcsname pgph@map@#1\endcsname\csname pgph@map@#1\endcsname\fi
458 }
459 \newcommand\pgph@ignorerule[2]{%
460   \edef\pgph@f{\pgph@resolve{#1}}\edef\pgph@t{\pgph@resolve{#2}}%
```

```

461 \ifx\pgph@f\@empty\else\ifx\pgph@t\@empty\else
462   \@namedef{pgph@ig@\pgph@f @\pgph@t}{}%
463 \fi\fi
464 }
465 \newcommand\pgph@excluderule[1]{%
466   \edef\pgph@x{\pgph@resolve{#1}}%
467   \ifx\pgph@x\@empty\else\@namedef{pgph@ex@\pgph@x}{}\fi
468 }

```

5.9 Emission

`\pgph@dotes` Escape the two characters that are special in a Graphviz double-quoted string, so a theorem name, number or citation note containing them does not corrupt the `.dot`: a backslash is doubled and a double quote is backslash-escaped (in that order). `\pgph@bschar` and `\pgph@dqchar` hold the catcode-12 backslash and double quote to search for; `xstring`'s expansion mode is saved and restored around the two substitutions.

```

469 \begingroup
470   \catcode'\|=0 \catcode'\="=12 \catcode'\|=12
471   \gdef\pgph@bschar{\}%
472   \gdef\pgph@dqchar{"}%
473 \endgroup
474 \newcommand\pgph@dotes[1]{%
475   \saveexpandmode\expandarg
476   \StrSubstitute{#1}{\pgph@bschar}{\pgph@bschar\pgph@bschar}[#1]%
477   \StrSubstitute{#1}{\pgph@dqchar}{\pgph@bschar\pgph@dqchar}[#1]%
478   \restoreexpandmode}

```

`\pgph@node` Emit one node line, honouring exclusions and per-environment styling. External (`cite:`) targets are emitted separately as a pass over edges. An *unnumbered* result (no number captured) is omitted: a results graph is about numbered statements, and unnumbered theorem-like environments are typically expository or repeated copies (e.g., a class-predefined `\spnewtheorem*{claim}`). `\proofgraphtrack{<env>}` overrides this, so a deliberately unnumbered environment is still drawn.

```

479 \newcommand\pgph@node[3]{%
480   \ifcsname pgph@ex@#1\endcsname\else
481     \edef\pgph@nm{\ifcsname pgph@num@#1\endcsname%
482       \csname pgph@num@#1\endcsname\fi}%
483     \ifx\pgph@nm\@empty
484       \ifcsname pgph@force@#2\endcsname
485         \pgph@emitnode{#1}{#2}{#3}%
486       \else
487         \@namedef{pgph@ex@#1}{}%
488         \fi
489       \else
490         \pgph@emitnode{#1}{#2}{#3}%
491         \fi
492     \fi
493 }
494 \newcommand\pgph@emitnode[3]{%
495   \edef\pgph@s{\ifcsname pgph@style@#2\endcsname
496     , \csname pgph@style@#2\endcsname\fi}%

```

```

497 \edef\pgph@dn{\ifcsname pgph@name@#1\endcsname
498   \csname pgph@name@#1\endcsname\else#3\fi}%
499 \edef\pgph@lbl{\pgph@dn\space\pgph@nm}%
500 \pgph@dotes\pgph@lbl
501 \immediate\write\pgph@out{%
502   \space\space"#1" [label="\pgph@lbl"\pgph@s];}%
503 \ifcsname pgph@rmap@#1\endcsname
504   \immediate\write\pgph@mapout{%
505     \string\pgph@nodemap{#1}{\csname pgph@rmap@#1\endcsname}}}%
506 \fi
507 }

```

\pgph@edge Emit one edge, resolving the target label to an id and applying the self-loop, ignore and exclude filters. Targets beginning with `cite:` denote external nodes.

```

508 \newcommand\pgph@edge[2]{%
509   \IfBeginWith{#2}{cite:}%
510     {\pgph@citeedge{#1}{#2}}%
511     {\pgph@plainedge{#1}{#2}}%
512 }
513 \newcommand\pgph@plainedge[2]{%
514   \edef\pgph@t{\pgph@resolve{#2}}%
515   \ifx\pgph@t\@empty\else
516     \ifcsname pgph@ex@#1\endcsname\else
517       \ifcsname pgph@ex@\pgph@t\endcsname\else
518         \ifcsname pgph@ig@#1@\pgph@t\endcsname\else
519           \ifboolexpr{ test {\ifnumequal{#1}{\pgph@t}}
520             and test {\ifdefstring{\pgph@selfloops}{remove}} }%
521             {}{\pgph@emitedge{#1}{n\pgph@t}}%
522           \fi\fi\fi
523   \fi
524 }

```

\pgph@emitedge The relation recorded is always “the result whose proof we are in (the first argument) uses the target (the second)”. The `direction` option chooses the arrow orientation: `direction=usedby` (default) draws *target to result*, so an arrow `a->b` reads “a is used by b” (i.e. b relies on a); `direction=uses` draws *result to target* (“a uses b”). The filters above work on the recorded relation, so they are unaffected by the chosen orientation. **\pgph@writeedge** deduplicates.

```

525 \newcommand\pgph@emitedge[2]{%
526   \ifdefstring{\pgph@direction}{uses}%
527     {\pgph@writeedge{#1}{#2}}%
528     {\pgph@writeedge{#2}{#1}}%
529 }
530 \newcommand\pgph@writeedge[2]{%
531   \ifcsname pgph@drawn@#1@#2\endcsname\else
532     \namedef{pgph@drawn@#1@#2}{}%
533     \immediate\write\pgph@out{\space\space"#1" -> "#2";}%
534   \fi
535 }

```

\pgph@citeedge A `cite:⟨slot⟩` target: recover the slot’s BibT_EX key, declare the external node once per slot, then draw the edge. The node *label* is built by **\pgph@setcitelabel**: it is the key by default, or, with `citelabel=tag`, the printed citation tag (the

“[42]” / “[Knu74]” text), recovered either from a tag recorded by `\pgph@blxrecord` under `biblatex` or from the `\b@⟨key⟩` macro that `\bibcite` writes to the `.aux` (so two runs are needed, and it falls back to the key when no tag is known, e.g., on the first run or with citation packages neither path covers). The slot’s `\cite` note, when present, is prepended as “*⟨note⟩*, ...”. The node-to-label map records the *key*, so all slots of one work hyperlink to the same bibliography entry.

```

536 \newcommand\pgph@citeedge[2]{%
537   \ifcsname pgph@ex@#1\endcsname\else
538     \StrBehind{#2}{cite:}[\pgph@slot]%
539   \edef\pgph@key{\csname pgph@cskey@\pgph@slot\endcsname}%
540   \ifcsname pgph@extnode@\pgph@slot\endcsname\else
541     \@namedef{pgph@extnode@\pgph@slot}{}%
542     \expandafter\pgph@setcitelabel\expandafter{\pgph@slot}%
543     \pgph@dotes\pgph@clabel
544     \immediate\write\pgph@out{%
545       \space\space"c@\pgph@slot"
546       [label="\pgph@clabel",\pgph@citestyle];}%
547     \immediate\write\pgph@mapout{%
548       \string\pgphnodemapcite{c@\pgph@slot}{\pgph@key}}%
549   \fi
550   \pgph@emitedge{#1}{c@\pgph@slot}%
551 \fi
552 }
553 \edef\pgph@obrace{\expandafter\@gobble\string\{}}
554 \newif\ifpgph@natnum
555 \newcommand\pgph@nil{}

```

`\pgph@citesanitize` neutralizes the wrappers and spacing cruft that citation labels carry, so that an `\edef` of `\b@⟨key⟩` yields plain text: `hyperref`’s `\hyper@@link` (keep its printed last argument) and the `\boxing/penalty/\ignorespaces` noise that `natbib` bakes into author lists.

```

556 \newcommand\pgph@citesanitize{%
557   \let\protect\@empty
558   \def\hyper@@link[##1]##2##3##4{##4}%
559   \let\unskip\@empty \let\ignorespaces\@empty \let\unhbox\@empty
560   \let\penalty\@gobble \let\@M\@empty \let\voidb@x\@empty
561   \let\nobreakspace\space \let~\space
562   \def\hbox##1{##1}\def\mbox##1{##1}%
563 }

```

A `natbib \b@⟨key⟩` is `{⟨num⟩}{⟨year⟩}{⟨short⟩}{⟨long⟩}`. In numbers mode the tag is the number; in author–year mode it is the short author list and the year.

```

564 \def\pgph@natpick#1#2#3#4\pgph@nil{%
565   \ifpgph@natnum\def\pgph@x{#1}\else\def\pgph@x{#3 #2}\fi}
566 \newcommand\pgph@natdo{\expandafter\pgph@natpick\pgph@x{}{}{}\pgph@nil}

```

`\pgph@natnumset` Which of the two `natbib` modes is in force is read off `\ifNAT@numbers`, by comparing it with `\iftrue`. That comparison *must* live in a macro of its own rather than inline in `\pgph@setcitelabel`, because the code around it can be skipped by a false conditional: skipping does not expand anything, it merely counts conditional tokens, and the `\iftrue` here – an operand of `\ifx`, not a conditional – would be counted as one more `\if` to be closed. The skip would then eat one `\fi` too many and run past the end of `\pgph@setcitelabel` into its caller, silently swallowing

the very writes that emit the citation node. Inside a macro the token is never seen by the skipping scanner.

```

567 \newcommand\pgph@natnumset{%
568   \pgph@natnumfalse
569   \@ifundefined{ifNAT@numbers}{-}{%
570     \expandafter\ifx\csname ifNAT@numbers\endcsname\iftrue
571       \pgph@natnumtrue\fi}%
572 }
```

`\pgph@blxrecord` biblatex writes no `\b@{key}`: its tags are assembled while the citation is typeset, from data the .bbl holds. We therefore record them as they go by, from `\AtEveryCitekey` (installed below), where the entry is the current one and its fields are readable. `labelnumber` carries the tag of the numeric styles and `labelalpha` that of the alphabetic ones; the author–year styles have neither, as their tag is a formatted name list rather than a field, and are left to the `key` fallback. A field that survives sanitizing with a backslash still in it is not plain text, and is likewise declined.

```

573 \newcommand\pgph@blxrecord{%
574   \begingroup
575     \pgph@citesanitize
576     \edef\pgph@blxkey{\thefield{entrykey}}%
577     \edef\pgph@x{\thefield{labelnumber}}%
578     \ifx\pgph@x\@empty \edef\pgph@x{\thefield{labelalpha}}\fi
579     \edef\pgph@dx{\detokenize\expandafter{\pgph@x}}%
580     \ifx\pgph@x\@empty\else
581       \IfSubStr\pgph@dx\@backslashchar{-}%
582       \pgph@gxdefcs{pgph@blxtag@pgph@blxkey}{\pgph@x}%
583     \fi
584   \endgroup
585 }

586 \newcommand\pgph@setcitelabel[1]{%
587   \edef\pgph@ckey{\csname pgph@cskey@#1\endcsname}%
588   \edef\pgph@cbase{\pgph@ckey}%
589   \ifdefstring{\pgph@citelabel}{tag}{%
590     \ifcsname pgph@blxtag@pgph@ckey\endcsname
591       \edef\pgph@cbase{\csname pgph@blxtag@pgph@ckey\endcsname}%
592     \else
593       \ifcsname b@pgph@ckey\endcsname
594         \global\let\pgph@cbaseg\relax
595         \begingroup
596           \pgph@citesanitize
597           \edef\pgph@x{\csname b@pgph@ckey\endcsname}%
598           \edef\pgph@dx{\detokenize\expandafter{\pgph@x}}%
599           \IfBeginWith\pgph@dx\pgph@obrace{%
600             \pgph@natnumset
601             \pgph@natdo
602             \edef\pgph@dx{\detokenize\expandafter{\pgph@x}}%
603           }-}%
604           \ifx\pgph@x\@empty\else
605             \IfSubStr\pgph@dx\@backslashchar{-}%
606             \global\let\pgph@cbaseg\pgph@x}%
607         \fi
```

```

608     \endgroup
609     \ifx\pgph@cbaseg\relax\else\edef\pgph@cbase{[\pgph@cbaseg]}\fi
610     \fi\fi
611   }{}%
612   \edef\pgph@clabel{%
613     \ifcsname pgph@citenote@#1\endcsname
614       \csname pgph@citenote@#1\endcsname, \fi
615     \pgph@cbase}%
616 }

```

`\pgph@render` `autorun` must not re-render a graph that has not changed. Rendering unconditionally is not merely wasteful: the image Graphviz produces is not guaranteed to be reproducible (its `cairo`-based outputs, `pdf` among them, carry a creation date), so an unconditional run makes the embedded image differ at every compilation. A build tool such as `latexmk` then sees an input file that never settles and reruns until it gives up, reporting that too many passes were needed.

We therefore render only when something relevant has changed. The signature of a rendering is the checksum of the `.dot` file just written, together with the engine and the format, which the `.dot` does not record; it is stored in the `.aux` by `\pgph@lastrender` and compared with the signature of the next run. Graphviz is run when the signature differs, when no signature is known yet (first run), when either output file is missing, and, as a safe fallback, when the engine provides no file checksum.

Both signatures are `\detokenized` before being compared. The checksum comes out of `\pdf@filemdfivesum` as characters of category *other*, while the same string read back from the `.aux` has its letters as category *letter*, so an `\ifx` on the raw strings would never see two runs as equal.

```

617 \let\pgph@prevsig\relax
618 \newcommand\pgph@lastrender[1]{\xdef\pgph@prevsig{\detokenize{#1}}}
619 \newcommand\pgph@render{%
620   \immediate\write18{\pgph@engine\space -T\pgph@format\space
621     \pgph@file.dot -o \pgph@file-proofgraph.\pgph@format}%
622   \immediate\write18{\pgph@engine\space -Tplain\space
623     \pgph@file.dot -o \pgph@file-proofgraph.plain}%
624 }
625 \newcommand\pgph@maybrender{%
626   \edef\pgph@sig{\ifdefined\pdf@filemdfivesum
627     \pdf@filemdfivesum{\pgph@file.dot}\fi}%
628   \ifx\pgph@sig\empty
629     \pgph@render
630   \else
631     \edef\pgph@sig{\pgph@sig-\pgph@engine-\pgph@format}%
632     \edef\pgph@sig{\expandafter\detokenize\expandafter{\pgph@sig}}%
633     \IfFileExists{\pgph@file-proofgraph.\pgph@format}%
634       {\IfFileExists{\pgph@file-proofgraph.plain}%
635         {\ifx\pgph@sig\pgph@prevsig\else\pgph@render\fi}%
636         {\pgph@render}}%
637     {\pgph@render}%

```

The signature is recorded whether or not we have just rendered: either way the output files on disk are the ones this `.dot` produces. The write must be `\immediate`, as `\pgph@emit` runs at end of document, where a deferred write would have no shipout left to carry it.

```

638 \if@filesw
639 \immediate\write\@auxout{\string\pgph@lastrender{\pgph@sig}}}%
640 \fi
641 \fi
642 }

```

\pgph@emit Open the file, resolve editing rules, write nodes then edges, close, and optionally run Graphviz.

```

643 \newwrite\pgph@out
644 \newwrite\pgph@mapout
645 \newcommand\pgph@emit{%
646 \immediate\openout\pgph@out=\pgph@file.dot\relax
647 \immediate\openout\pgph@mapout=\pgph@file-proofgraph.map\relax
648 \immediate\write\pgph@out{digraph proofgraph \pgph@ob}%
649 \immediate\write\pgph@out{\space\space rankdir="\pgph@rankdir";}%
650 \begingroup
651 \pgph@ignores
652 \pgph@excludes
653 \pgph@nodes
654 \pgph@edges
655 \endgroup
656 \immediate\write\pgph@out{\pgph@cb}%
657 \immediate\closeout\pgph@out
658 \immediate\closeout\pgph@mapout
659 \ifpgph@autorun\pgph@maybrender\fi
660 }

```

The emission is registered at `\begin{document}` rather than at load time so that it is added to the end-document hook *after* packages loaded later (notably `apxproof`, whose appendix, with the deferred proofs and their references, is built at end of document). This way the graph is written only once that material has been typeset, and the emission does not disturb the verbatim replay of deferred proofs.

```

661 \AtBeginDocument{\AtEndDocument{\pgph@emit}}

```

If TikZ is available, load its `calc` library now (at `\begin{document}`, where category codes are normal) so that the hyperlink overlay in `\proofgraph` can use it without reading a file mid-document.

```

662 \AtBeginDocument{\@ifpackageloaded{tikz}{\usetikzlibrary{calc}}{}}

```

\pgphnodemap One entry of the node-to-target map written by `\pgph@emit`: it records, for a Graphviz node identifier, what the node should link to. `\pgphnodemap` records the user *label* of the result a node represents; `\pgphnodemapcite` records the BibTeX *key* of an external citation node, so that (when the cited work is in the same document) the node can link to its bibliography entry, `hyperref`'s `cite.<key>` destination. The map is read back when the graph is embedded, to turn each node into a hyperlink.

```

663 \newcommand\pgphnodemap[2]{\global\@namedef{pgph@lbl@#1}{#2}}
664 \newcommand\pgphnodemapcite[2]{\global\@namedef{pgph@cbl@#1}{#2}}

```

\proofgraph Include the rendered graph (from a previous run) in `autorun` mode. When `hyperref` and `TikZ` are both available (and `hyperlinks` is not switched off), each node is made into an internal hyperlink to what it represents: a result node links to the result (its `\label`), and an external citation node (from the `cite` option) links

to the cited work's bibliography entry, `hyperref's cite.<key>` destination, when that work is cited in the same document. `Graphviz` produces a `.pdf` whose link annotations would be lost by `\includegraphics` (they sit inside a form `XOBJECT`), so instead we overlay transparent link areas at the node positions, which we read from the `-Tplain` output. Without those packages we fall back to a plain inclusion.

```

665 \newif\ifpgph@canlink
666 \newif\ifpgph@link
667 \newsavebox\pgph@imgbox
668 \newsavebox\pgph@natbox
669 \newread\pgph@plainin
670 \def\pgph@stop{}
671 \newcommand\proofgraph[1][]{%
672   \ifundefined{includegraphics}%
673     {\PackageError{proofgraph}{\string\proofgraph\space requires the
674       graphicx package}{Add \string\usepackage{graphicx} to your preamble
675       to embed the rendered graph.}}%
676     {\IfFileExists{\pgph@file-proofgraph.\pgph@format}%
677       {\pgph@includegraph{#1}}%
678       {\PackageWarning{proofgraph}{Rendered graph
679         \pgph@file-proofgraph.\pgph@format\space not found; compile with
680         shell-escape and the autorun option, then re-run}}}%
681 }
682 \newcommand\pgph@includegraph[1]{%
683   \pgph@canlinkfalse
684   \ifpgph@hyperlinks
685     \ifpackageloaded{hyperref}%
686       {\ifpackageloaded{tikz}%
687         {\IfFileExists{\pgph@file-proofgraph.plain}%
688           {\pgph@canlinktrue}{}}}%
689     \fi
690     \ifpgph@canlink
691       \pgph@graphlinked{#1}%
692     \else
693       \includegraphics[#1]{\pgph@file-proofgraph.\pgph@format}%
694     \fi
695 }

```

The linked version. We read the node-to-target map under safe category codes (a label may contain `_` or, under `babel`, an active `:`), measure the rendered image, place it in a `TikZ` node, then read the `-Tplain` file and drop one transparent link box on each mapped node. `Graphviz` draws the graph inset by a margin, so the image is larger than the layout bounding box the `-Tplain` coordinates live in; we recover that margin by also measuring the image at its *natural* size and comparing it with the `-Tplain` graph size (the `graph` line gives it in inches, hence the factor 72). Node positions are then expressed as fractions of the natural image, so they line up with the drawn nodes whatever size the image is finally scaled to.

```

696 \newcommand\pgph@graphlinked[1]{%
697   \begingroup
698     \catcode'\:=12 \catcode'\_ =12 \catcode'\;=12 \catcode'\!=12
699     \catcode'\?=12 \catcode'\&=12 \catcode'\^=12 \catcode'\~=12
700     \InputIfFileExists{\pgph@file-proofgraph.map}{}{%
701   \endgroup
702   \sbox\pgph@imgbox{%

```

```

703 \includegraphics[#1]{\pgph@file-proofgraph.\pgph@format}}%
704 \sbox\pgph@natbox{%
705 \includegraphics{\pgph@file-proofgraph.\pgph@format}}%
706 \edef\pgph@imgw{\the\wd\pgph@imgbox}%
707 \edef\pgph@imgh{\the\dimexpr\ht\pgph@imgbox+\dp\pgph@imgbox\relax}%
708 \edef\pgph@natw{\the\wd\pgph@natbox}%
709 \edef\pgph@nath{\the\dimexpr\ht\pgph@natbox+\dp\pgph@natbox\relax}%
710 \begin{tikzpicture}
711 \node[inner sep=\z@,outer sep=\z@](pgph@P){\usebox\pgph@imgbox};
712 \openin\pgph@plainin=\pgph@file-proofgraph.plain\relax
713 \pgph@loopplain
714 \closein\pgph@plainin
715 \end{tikzpicture}%
716 }
717 \newcommand\pgph@loopplain{%
718 \unless\ifeof\pgph@plainin
719 \read\pgph@plainin to \pgph@line
720 \pgph@procline
721 \expandafter\pgph@loopplain
722 \fi
723 }
724 \newcommand\pgph@procline{%
725 \IfBeginWith{\pgph@line}{graph }%
726 {\expandafter\pgph@dograph\pgph@line\pgph@stop}%
727 {\IfBeginWith{\pgph@line}{node }%
728 {\expandafter\pgph@donode\pgph@line\pgph@stop}{}}%
729 }
730 \def\pgph@dograph graph #1 #2 #3 #4\pgph@stop{%
731 \def\pgph@gw{#2}\def\pgph@gh{#3}}
732 \def\pgph@donode node #1 #2 #3 #4 #5 #6\pgph@stop{%
733 \StrDel{#1}{"}[\pgph@nid]%
734 \pgph@linkfalse
735 \ifcsname pgph@lbl@\pgph@nid\endcsname
736 \edef\pgph@tgt{\csname pgph@lbl@\pgph@nid\endcsname}%
737 \edef\pgph@thislink{\noexpand\hyperref[\pgph@tgt]}%
738 \pgph@linktrue
739 \else
740 \ifcsname pgph@clbl@\pgph@nid\endcsname
741 \edef\pgph@tgt{\csname pgph@clbl@\pgph@nid\endcsname}%
742 \edef\pgph@thislink{\noexpand\hyperlink{cite.\pgph@tgt}}%
743 \pgph@linktrue
744 \fi
745 \fi
746 \ifpgph@link
747 % Per-side Graphviz margin: half the natural-minus-layout size.
748 \pgfmathsetmacro\pgph@mx{(\pgph@natw-\pgph@gw*72)/2}%
749 \pgfmathsetmacro\pgph@my{(\pgph@nath-\pgph@gh*72)/2}%
750 % Node centre as a fraction of the natural image.
751 \pgfmathsetmacro\pgph@fx{(\pgph@mx+#2*72)/\pgph@natw}%
752 \pgfmathsetmacro\pgph@fy{(\pgph@my+#3*72)/\pgph@nath}%
753 \pgfmathsetlengthmacro\pgph@bw{#4*72/\pgph@natw*\pgph@imgw*0.92}%
754 \pgfmathsetlengthmacro\pgph@bh{#5*72/\pgph@nath*\pgph@imgh*0.9}%
755 \node[anchor=center]
756 at ($(\pgph@P.south west)!\pgph@fx!(pgph@P.south east)$)%

```

```

757      !\pgph@fy!($(\pgph@P.north west)!\pgph@fx!(\pgph@P.north east)$)$)
758      {\pgph@thislink{\phantom{\rule{\pgph@bw}{\pgph@bh}}}};%
759    \fi
760  }
761 </package>

```

Index

Numbers written in *italic* refer to the page where the corresponding entry is described; numbers underlined refer to the code line of the definition; numbers in roman refer to the code lines where the entry is used.

Symbols		C	
\!	698	\catcode	34, 35, 470, 698, 699
\"	470	\closein	714
\&	699	\Cref	292
\:	698	\cref	291
\;	698	\cs	17
\?	699	\csundef	92, 100
\@M	560	D	
\@Opargbegintheorem	281, 282, 283	\DeclareBoolOption	21, 26, 28, 29
\@auxout	56, 639	\DeclareStringOption	20, 22, 23, 24, 25, 27, 30
\@backslashchar	581, 605	\detokenize	300, 369, 579, 598, 602, 618, 632
\@begintheorem	144, 145, 147, 151	\dimexpr	707, 709
\@currentlabel	193, 207	\dp	707, 709
\@ifpackageloaded	5, 329, 413, 662, 685, 686	E	
\@ifundefined	6, 10, 298, 569, 672	\end	715
\@namedef	49, 63, 93, 101, 107, 190, 453, 462, 467, 487, 532, 541, 663, 664	\endproof	7, 8, 11
\@opargbegintheorem	146, 155, 156, 157, 270, 271, 272	\expandarg	475
\@protected@testopt	300	\ExplSyntaxOff	18
\@spbegintheorem	161, 162, 163	\ExplSyntaxOn	16
\@spopargbegintheorem	166, 167, 168, 275, 276, 277	H	
\[	34	\hbox	178, 294, 562
\]	470	\ht	707, 709
\{	35, 553	\hyper@@link	558
\}	35, 471	\hyperlink	742
\]	34	\hyperref	737
\^	699	I	
_	698	\if@filesw	638
\	470	\IfBeginWith	61, 509, 599, 725, 727
\~	699	\IfBooleanTF	237
A		\ifboolexpr	324, 519
\apxproofhook	333	\ifdefstring	324, 412, 520, 526, 589
\AtBeginDocument	127, 328, 661, 662	\ifeof	718
\AtBeginEnvironment	108, 303	\IfFileExists	633, 634, 676, 687
\AtEndDocument	661	\ifnumequal	519
\AtEndPreamble	411	\ifpgph@apxproof	320
\AtEveryCitekey	413	\ifpgph@autorun	659
\autoref	292	\ifpgph@canlink	665, 690
\axp@forward@link	186, 206	\ifpgph@cite	258
B		\ifpgph@hyperlinks	684
\begin	710	\ifpgph@link	666, 746
		\ifpgph@natnum	554, 565
		\ifpgph@statements	141

<code>\IfSubStr</code>	301, 581, 605	<code>\pgph@cap@cite@opt</code>	348, 349
<code>\iftrue</code>	570	<code>\pgph@cap@cite@opti</code>	350, 351
<code>\ignorespaces</code>	559	<code>\pgph@cap@cite@optii</code>	350, 353
<code>\includegraphics</code>	693, 703, 705	<code>\pgph@cap@cite@x</code>	344, 345
<code>\InputIfFileExists</code>	700	<code>\pgph@cap@cite@gen</code>	342, 399
J		<code>\pgph@cap@citerec</code>	
<code>\jobname</code>	32		352, 354, 355, 356, 432
L		<code>\pgph@cap@Cref</code>	229
<code>\label</code>	139, 140	<code>\pgph@cap@ceref</code>	228
M		<code>\pgph@cap@eqref</code>	232
<code>\mbox</code>	562	<code>\pgph@cap@ref</code>	227
<code>\meaning</code>	299	<code>\pgph@cap@vref</code>	234
N		<code>\pgph@cap@zceref</code>	235
<code>\newcount</code>	39, 40	<code>\pgph@cb</code>	37, 656
<code>\NewDocumentCommand</code>	235, 287, 432, 433	<code>\pgph@cbase</code>	588, 591, 609, 615
<code>\newif</code>	320, 554, 665, 666	<code>\pgph@cbaseg</code>	594, 606, 609
<code>\newread</code>	669	<code>\pgph@citecmd</code>	343, 347
<code>\newsavebox</code>	667, 668	<code>\pgph@citecmds</code>	382
<code>\newtheorem</code>	72, 73	<code>\pgph@citeedge</code>	443, 510, 536
<code>\nobreakspace</code>	561	<code>\pgph@citelabel</code>	412, 589
<code>\node</code>	711, 755	<code>\pgph@citelabeledge</code>	432
<code>\noexpand</code> ..	136, 216, 347, 443, 737, 742	<code>\pgph@citesanitize</code> ..	366, 556, 575, 596
O		<code>\pgph@citeslot</code>	360, 364, 441
<code>\openin</code>	712	<code>\pgph@citeslotcnt</code>	40, 373, 374
P		<code>\pgph@citestyle</code>	454, 455, 546
<code>\PackageError</code>	673	<code>\pgph@ckey</code> ..	587, 588, 590, 591, 593, 597
<code>\PackageWarning</code>	678	<code>\pgph@clabel</code>	543, 546, 612
<code>\pdf@filemdfivesum</code>	626, 627	<code>\pgph@class@endproof</code>	7, 11
<code>\penalty</code>	560	<code>\pgph@class@proof</code>	7, 11
<code>\pgfmathsetlengthmacro</code>	753, 754	<code>\pgph@ccone</code>	393, 394
<code>\pgfmathsetmacro</code> ..	748, 749, 751, 752	<code>\pgph@conename</code>	392, 393
<code>\pgph@adddge</code>	215, 224, 361	<code>\pgph@curproof</code> ..	46, 176, 221, 224,
<code>\pgph@apxinit</code>	329, 332		261, 266, 335, 336, 357, 361, 418
<code>\pgph@apxproofcapture</code>	332	<code>\pgph@curresult</code>	45, 134,
<code>\pgph@apxprooftrue</code>	329		136, 174, 176, 183, 188, 190,
<code>\pgph@auxmap</code>	47		194, 201, 202, 203, 208, 261, 266
<code>\pgph@beginproof</code>	227, 303, 309	<code>\pgph@curslot</code>	
<code>\pgph@beginresult</code>	108, 129		361, 371, 374, 375, 376, 378, 443
<code>\pgph@beginstatement</code>	141, 265	<code>\pgph@defaultenvs</code>	85, 128
<code>\pgph@bh</code>	754, 758	<code>\pgph@defaulttexclude</code>	88, 104
<code>\pgph@blxkey</code>	576, 582	<code>\pgph@direction</code>	526
<code>\pgph@blxrecord</code>	413, 573	<code>\pgph@dn</code>	497, 499
<code>\pgph@bschar</code>	476, 477	<code>\pgph@dograph</code>	726, 730
<code>\pgph@bw</code>	753, 758	<code>\pgph@dohook</code>	105, 113, 115
<code>\pgph@candidates</code> ..	58, 64, 65, 67, 122, 123	<code>\pgph@donode</code>	728, 732
<code>\pgph@canlinkfalse</code>	683	<code>\pgph@dotesc</code>	469, 500, 543
<code>\pgph@canlinktrue</code>	688	<code>\pgph@dqchar</code>	477
<code>\pgph@cap@autoref</code>	230	<code>\pgph@dx</code> ..	579, 581, 598, 599, 602, 605
<code>\pgph@cap@cite@noopt</code>	348, 355	<code>\pgph@edge</code>	216, 508
		<code>\pgph@edges</code>	42, 217, 422, 434, 654
		<code>\pgph@ef</code> ..	425, 426, 429, 437, 438, 443
		<code>\pgph@emit</code>	643
		<code>\pgph@emitedge</code>	521, 525, 550
		<code>\pgph@emitnode</code>	485, 490, 494

\pgph@engine	620, 622, 631	\pgph@natpick	564, 566
\pgph@excluderule	451, 465	\pgph@natw	708, 748, 751, 753
\pgph@excludes	44, 451, 652	\pgph@nid	733, 735, 736, 740, 741
\pgph@f	460, 461, 462	\pgph@nil	555, 564, 566
\pgph@file	32, 621, 623, 627, 633, 634, 646, 647, 676, 679, 687, 693, 700, 703, 705, 712	\pgph@nm	481, 483, 499
\pgph@format	620, 621, 631, 633, 676, 679, 693, 703, 705	\pgph@node	136, 479
\pgph@fx	751, 756, 757	\pgph@nodes	41, 130, 653
\pgph@fy	752, 757	\pgph@nt@plain	73, 74
\pgph@gh	731, 749	\pgph@nt@star	73, 76
\pgph@gnotestr	367, 369, 377, 378	\pgph@ob	36, 648
\pgph@graphlinked	691, 696	\pgph@obrace	553, 599
\pgph@gw	731, 748	\pgph@one	90, 91, 92, 93, 94, 98, 99, 100, 101, 119, 120, 121, 123, 124, 222, 223, 322, 323, 358, 359, 427, 428, 439, 440
\pgph@gxdefcs	340, 375, 376, 378, 582	\pgph@orig@bt	145, 149, 152
\pgph@hookcite	340	\pgph@orig@newtheorem	72, 75, 76
\pgph@hooknames	129	\pgph@orig@obt	156, 159
\pgph@hookproofhead	263, 269	\pgph@orig@obtp	282, 284
\pgph@idcnt	39, 133, 134	\pgph@orig@obtp	271, 273
\pgph@ignorerule	448, 459	\pgph@orig@spbt	162, 164
\pgph@ignores	43, 448, 651	\pgph@orig@spnewtheorem	78, 82, 83
\pgph@imgbox	667, 702, 706, 707, 711	\pgph@orig@spobt	167, 170
\pgph@imgh	707, 754	\pgph@orig@spobtp	276, 279
\pgph@imgw	706, 753	\pgph@out	501, 533, 544, 643, 646, 648, 649, 656, 657
\pgph@includegraph	677, 682	\pgph@pe	323, 324, 325
\pgph@install	239, 252, 253, 254	\pgph@plainedge	429, 511, 513
\pgph@installcapture	177, 256, 262, 267, 337	\pgph@plainin	669, 712, 714, 718, 719
\pgph@installcite	258, 382	\pgph@prependnode	129, 137
\pgph@installciteone	394, 396	\pgph@prevsig	617, 618, 635
\pgph@installrefs	251, 257	\pgph@procline	720, 724
\pgph@k	223, 224, 359, 360, 428, 429, 440, 441	\pgph@proofenvs	315, 322
\pgph@key	539, 548	\pgph@proofmeaning	299, 301
\pgph@labeledge	422, 424	\pgph@proofopt	310, 313
\pgph@lastrender	617	\pgph@rankdir	649
\pgph@lbl	499, 500, 502	\pgph@recordref	220, 227, 228, 229, 231, 233, 234, 236, 415
\pgph@line	719, 725, 726, 727, 728	\pgph@recordresult	129
\pgph@linkfalse	734	\pgph@redefproof	302, 306
\pgph@linktrue	738, 743	\pgph@register	58, 75, 82, 94, 121
\pgph@loopplain	713, 717, 721	\pgph@render	617
\pgph@maplabel	139, 140, 200	\pgph@replay	346, 352, 354, 355
\pgph@mapout	504, 547, 644, 647, 658	\pgph@resolve	335, 425, 437, 456, 460, 466, 514
\pgph@maybehook	111, 124	\pgph@resolverregistration	118, 128
\pgph@maybrender	617, 659	\pgph@s	495, 502
\pgph@mx	748, 751	\pgph@s@saved@autoref	231
\pgph@my	749, 752	\pgph@s@saved@Cref	229
\pgph@natbox	668, 704, 708, 709	\pgph@s@saved@ceref	228
\pgph@natdo	566, 601	\pgph@s@saved@eqref	233
\pgph@nath	709, 749, 752, 754	\pgph@s@saved@ref	227
\pgph@natnumfalse	568	\pgph@s@saved@vref	234
\pgph@natnumset	567, 600	\pgph@s@saved@zceref	237
\pgph@natnumtrue	571		

<code>\pgph@savetlabel</code>	139, 213	<code>\proofgraphstylecite</code>	7, 455
<code>\pgph@scanproofarg</code> .	273, 278, 284, <u>287</u>	<code>\proofgraphtrack</code>	6, 13, <u>85</u>
<code>\pgph@scanrefs</code>	148, 158, 169, <u>173</u>	<code>\proofgraphuntrack</code>	7, 13, <u>85</u> , 318
<code>\pgph@selfloops</code>	520	<code>\proofof</code>	6, 287, 291, 292, 416
<code>\pgph@setcitelabel</code>	536	<code>\protect</code>	185, 205, 290, 557
<code>\pgph@setmap</code>	50, 201	<code>\protected</code>	227, 228, 229, 230, 232, 234
<code>\pgph@sig</code>	369, 370, 371, 375, 626, 628, 631, 632, 635, 639	<code>\protected@edef</code>	135, 216
<code>\pgph@slot</code>	538, 539, 540, 541, 542, 545, 548, 550	<code>\protected@write</code>	56
<code>\pgph@spnt@plain</code>	80, 81		
<code>\pgph@spnt@star</code>	80, 83	R	
<code>\pgph@stop</code> ...	670, 726, 728, 730, 732	<code>\read</code>	719
<code>\pgph@t</code>	460, 461, 462, 514, 515, 517, 518, 519, 521	<code>\ref</code>	291
<code>\pgph@testoptstr</code>	300, 301	<code>\restoreexpandmode</code>	478
<code>\pgph@tgt</code>	736, 737, 741, 742	<code>\rule</code>	758
<code>\pgph@thislink</code>	737, 742, 758		
<code>\pgph@thisname</code> ..	60, 61, 62, 63, 65, 67	S	
<code>\pgph@tmp</code>	135, 137	<code>\saveexpandmode</code>	475
<code>\pgph@tmpce</code>	442, 444	<code>\sbox</code>	702, 704
<code>\pgph@tmpedge</code>	216, 218	<code>\setbox</code>	178, 294
<code>\pgph@tmpphn</code>	191, 192	<code>\SetupKeyvalOptions</code>	19
<code>\pgph@tmpid</code>	51, 52, 53, 54, 56	<code>\spnewtheorem</code>	77, 78, 79
<code>\pgph@tmpname</code>	187, 189	<code>\StrBehind</code>	538
<code>\pgph@tmpnum</code>	193, 195, 207, 209	<code>\StrDel</code>	733
<code>\pgph@trimsp</code>	16, 91, 99, 223, 323, 359, 393, 428, 440	<code>\StrSubstitute</code>	476, 477
<code>\pgph@wrapproof</code>	287		
<code>\pgph@wrapproofone</code>	297, 325	T	
<code>\pgph@wrapproofs</code>	321, 330	<code>\thefield</code>	576, 577, 578
<code>\pgph@writeedge</code>	525	<code>\tl</code>	17
<code>\pgph@x</code>	466, 467, 565, 566, 577, 578, 579, 580, 582, 597, 598, 602, 604, 606		
<code>\pgph@zcproofof</code>	287, 293	U	
<code>\pgph@nodemap</code>	505, <u>663</u>	<code>\unexpanded</code>	130
<code>\pgph@nodemapcite</code>	548, <u>663</u>	<code>\unhbox</code>	559
<code>\phantom</code>	758	<code>\unless</code>	718
<code>\ProcessKeyvalOptions</code>	31	<code>\unskip</code>	559
<code>\proof</code>	7, 8, 11	<code>\usebox</code>	711
<code>\proofgraph</code>	8, <u>665</u>	<code>\usepackage</code>	674
<code>\proofgraphcitecommand</code>	7, <u>382</u>	<code>\uses</code>	6, 415
<code>\proofgraphciteedge</code>	6, 24, <u>432</u>	<code>\usescite</code>	6, 24, <u>432</u>
<code>\proofgraphhedge</code>	6, 421	<code>\usetikzlibrary</code>	662
<code>\proofgraphexclude</code>	7, 450		
<code>\proofgraphignore</code>	7, 447	V	
<code>\proofgraphproofenv</code>	7, 21, <u>315</u>	<code>\voidb@x</code>	560
<code>\proofgraphstyle</code>	7, 453		
		W	
		<code>\wd</code>	706, 708
		Z	
		<code>\z@</code>	178, 294, 711
		<code>\zcref</code>	293

Change History

v0.1.0		option, on by default, turns this off	1
General: Initial version	1		
v1.0.0		Capture, with the <code>cite</code> option, every citation command of <code>natbib</code> and <code>biblatex</code> , such as <code>textcite</code> and <code>parencite</code> , down to those printing a mere fragment of a citation such as <code>citeauthor</code> , and add <code>proofgraphcitecommand</code> to capture further ones	1
General: First released version . . .	1		
v1.0.1		Emit citation nodes again when <code>citelabel=tag</code> finds no tag to use. A stray <code>iftrue</code> , counted as a conditional while a false conditional was being skipped over, made the skip overshoot and swallow the writes emitting the node, which Graphviz then drew from its edges alone: unstyled, labelled by its internal name and without a hyperlink. It affected every first run, and <code>biblatex</code> documents always	1
General: Capture <code>citet</code> and <code>citep</code> (including starred variants) like <code>cite</code> when they are defined . . .	1		
Fix an infinite loop when, with the <code>cite</code> option, a citation occurred in a proof nested within another proof: the capture no longer reinstalls over itself	1		
Support, with the <code>cite</code> option, the two-optional-argument citation form <code>[pre][post]{key}</code> of <code>natbib</code> and <code>biblatex</code> ; it previously broke compilation with confusing errors inside proofs . .	1		
v1.0.1-dev		Recover the citation tag of the numeric and alphabetic <code>biblatex</code> styles for <code>citelabel=tag</code> , which records no citation tag in the <code>.aux</code> for it to read	1
General: Add <code>proofgraphproofenv</code> , declaring further environments to be treated as proofs, for a document whose proofs do not live in an environment named <code>proof</code>	1	Require a LaTeX2e format of October 2020 or later, which the use of <code>expl3</code> and <code>NewDocumentCommand</code> already implied	1
Add <code>usescite</code> and <code>proofgraphciteedge</code> , recording a dependency on cited work by hand, for a proof that does not cite it visibly and for a result stated without a proof to hold a citation	1	Trim the spaces after the commas of the lists given to <code>proofgraphtrack</code> and <code>proofgraphuntrack</code> ; every name but the first was silently ignored	1
Capture <code>zcref</code> of <code>zref-clever</code> , in its starred, optional-argument and multiple-label forms, like the other cross-referencing commands	1	With <code>autorun</code> , run Graphviz only when the graph, the engine or the format has changed. Rendering at every compilation kept producing a different image, since Graphviz stamps a creation date into its <code>cairo</code> -based outputs, and <code>latexmk</code> would then rerun until it reported that too many passes were needed	1
Capture citations, and not only cross-references, in the optional title of a result	1		
Capture cross-references and citations in the body of a result and not only in its proof and its optional title, a statement resting on an earlier result being a dependency like any other; the new <code>statements</code>			